Epiphany

Binary Format

A companion to the Core Specification

Version 0.21.0 — Schema major 5: expression and text

Normative for the byte layouts it defines

Contents

1 About This Companion 4
- 1.1 Relationship to the Core Specification and the Operation Catalog . . . 5
- 1.2 Conformance . . . 5

2 Encoding Conventions 7
- 2.1 The Three Encoding Regimes . . . 7
- 2.2 Primitive Composition Rules . . . 8

3 Identifiers and Derivations 10
- 3.1 The 128-Bit Identifier Family . . . 10
- 3.2 `TypedObjectId` . . . 10
- 3.3 System-Derived Counters and Digest Truncations . . . 11
- 3.4 The Domain-Tag Registry . . . 12
- 3.5 `SnapshotId` . . . 12

4 Primitive Value Encodings 14
- 4.1 `RationalTime` . . . 15

5 Graph Value Layouts 16
- 5.1 The `Score` Envelope . . . 16
- 5.2 Composition Rules as Normative Encoding Facts . . . 17
- 5.3 Discriminant Tables . . . 18
  - 5.3.1 C-style enums (one byte, no payload) . . . 18
  - 5.3.2 Tagged unions (one byte, then the variant payload) . . . 18
- 5.4 Representative Complete Layouts . . . 19
  - 5.4.1 `RationalTime` . . . 19
  - 5.4.2 `TimeAnchor` . . . 19
  - 5.4.3 `Pitch` . . . 20
  - 5.4.4 `Event` . . . 20
  - 5.4.5 `Slur` . . . 20
- 5.5 The `CanonicalValue` Seam . . . 20
- 5.6 The Frozen-Layout Rule . . . 21

6 Operation Wire Forms 22
- 6.1 The Operation Envelope . . . 22
- 6.2 `OperationPayload` and the Meta-Operations . . . 23
- 6.3 `OperationKind`: the Primitive Wire Discriminants . . . 23
- 6.4 `OperationKindTag`: a Separate Discriminant Space . . . 25

6.5 Stamps and Causal Contexts . . . 26
6.6 Effects, Conflicts, Anomalies, and Materialized State . . . 27
6.6.1 `MaterializedState` . . . 28
6.7 v0 Payloads Have No Wire Form . . . 28

7 Bundle Physical Layout 29
7.1 The Fixed Header (64 bytes) . . . 29
7.2 The Superblock Slots (256 bytes each) . . . 30
7.3 Chunks . . . 30
7.3.1 `ChunkRef` (95 bytes) . . . 31
7.4 Operation-Envelope Block Payloads . . . 32
7.5 The Manifest . . . 32
7.6 The Operation Index . . . 34

8 Extension Declaration Blobs and Edit Barriers 35
8.1 Framing . . . 35
8.2 `EditBarrier` . . . 36

9 Schema Evolution 38
9.1 The Chunk-Level Gate . . . 38
9.2 What "Additive" Means Here . . . 39
9.3 Migration Discipline . . . 40
9.4 Schema Major 1 . . . 40
9.4.1 Where the changed fields reach . . . 40
9.4.2 Cross-major reader behaviour . . . 41
9.4.3 The accept-set gate . . . 42
9.4.4 Changed and new value layouts . . . 42
9.4.5 Length-prefix unification (partial) . . . 42
9.4.6 Migration from major 0 . . . 42
9.5 Schema Major 2 . . . 43
9.5.1 Where the changed fields reach . . . 43
9.5.2 Cross-major reader behaviour . . . 45
9.5.3 Changed value layouts . . . 45
9.5.4 New leaf-type layouts . . . 45
9.5.5 Migration from major 1 . . . 47
9.6 Schema Major 3 . . . 47
9.6.1 Where the changed fields reach . . . 48
9.6.2 Cross-major reader behaviour . . . 48
9.6.3 Changed value layout . . . 48
9.6.4 New leaf-type layouts . . . 49
9.6.5 Migration from major 2 . . . 49
9.7 Schema Major 4 . . . 49
9.7.1 Where the changed fields reach . . . 50
9.7.2 Cross-major reader behaviour . . . 50
9.7.3 Changed value layout . . . 50
9.7.4 New leaf-type layouts . . . 50
9.7.5 Migration from major 3 . . . 51
9.8 Schema Major 5 . . . 51

9.8.1 Where the changed fields reach . . . 51
9.8.2 Cross-major reader behaviour . . . 51
9.8.3 Changed value layout . . . 52
9.8.4 New leaf-type layouts . . . 52
9.8.5 Appended variants . . . 53
9.8.6 Migration from major 4 . . . 53

10 Non-Canonical Pinned Encodings 54
10.1 `LayoutObjectId` (`MUSCLOID`) . . . 54
10.2 `ResolvedLayoutIR` Canonical Output . . . 54
10.3 Layout Caches and Font Metrics . . . 55

11 Golden Anchor Registry 56
11.1 The Decode Vector Corpus . . . 58

12 Revision History 60

Chapter 1

About This Companion

The *Binary Format* document is a companion to the Epiphany Core Specification. It fulfils the core specification's delegation in Chapter 8, Binary Format Companion (the `sec:format:binary` section): the byte-level encoding — integer conventions, exact field widths, record alignment, endianness, string encoding details, struct layouts — is delivered here. In the core specification's words: *"The Binary Format document is normative; an implementation cannot conform to the file format specification without conforming to the Binary Format specification."*

This release (v0.1.0) delivers the full **K-independent** encoding surface — identifier and derivation layouts, primitive value encodings, the whole-`Score` composite value codec, the operation-layer wire forms, the bundle physical layout, and the extension-declaration blob forms — *plus* the **K0/M2 operation-payload wire framing**. The Operation Catalog's K0 set (the representative primitives and the M2 broad-K0 groups, catalog version 0.4.0) now exists in ratified form, so the historical "waits for Agent K" clause in the Phase-2 charter is discharged: every operation kind the catalog defines has its literal wire form pinned in Chapter 6.

This document does *not* cover:

- the canonical s-expression form — that is the *Text Projection* companion's (version 0.1.0, which now exists);
- per-profile feature lists — the *Profile Conformance* companion's;
- the Chapter-4 tuning-catalog values (pitch-space and tuning-system registries), which are undelivered Track-C work and have no wire form yet;
- the wire-format fuzzer, an *implementation* deliverable tracked in the Phase-2 process documents rather than part of this document's normative text; Chapter 11 records the golden anchors any such harness must reproduce.

It *does* cover the cross-implementation decoder test, which is no longer deferred: Section 11.1 ratifies the literal-byte vector corpus and the accept/reject verdict a conforming decoder owes it.

1.1 Relationship to the Core Specification and the Operation Catalog

This companion *inherits* — it never re-derives — the core specification's ratified convention baseline: core specification Chapter 8, Binary Format Companion, requirement `req:format:codec-conventions`, together with the Canonical Byte-Layout Reference appendix (`app:bytes`). That appendix consolidates every discriminant table, derivation preimage, and primitive encoding Pass 11 and Pass 12 ratified, precisely so that this document could import it as a starting point rather than recovering the layouts from the crates. Chapter 3 therefore reproduces only *compact* normative tables, each with a citation back to the appendix row and requirement that governs it; where this document and a ratified core requirement disagree, **the core requirement governs** and the discrepancy is a defect in this document.

What this document adds is the completion of the appendix's deferred list (core specification, Layouts deferred to the companions, `sec:bytes:deferred`): the `OperationKind` and `OperationKindTag` discriminants and full per-operation payload encodings; the full composite struct layouts (whole-`Score`, operation envelopes, the manifest body's field order); schema-version wire evolution; and the varint question, which Chapter 2 settles.

The division of labour with the *Operation Catalog* is exact: the catalog fixes each operation's **field set and order** (Operation Catalog, Per-Primitive Schema Template) and its semantics; this document pins the **bytes** of those fields. Chapter 6 cites the catalog section for every payload rather than restating what the fields mean. Likewise, graph *types* are the core specification's Chapters 2–5, and the structural layer of the bundle — what a header, superblock, chunk graph, or manifest *is* and how it behaves — is core Chapter 8; this document specifies only how those structures' bits are laid out.

Rationale

Versioning. This companion is versioned independently of the core specification (independent semver), like the Operation Catalog — but tied tighter to it: the binary-format major version tracks the core specification's major version (both are pre-1.0 working drafts today, hence v0.1.0), while minor versions diverge freely. Byte layouts change on a slower, more deliberate cadence than prose; a core-spec minor revision that touches no wire fact requires no revision here, and an additive discriminant append here (Chapter 9) requires no core-spec revision.

1.2 Conformance

The byte layouts in this document are **normative**. An implementation conforms to the Epiphany binary format if and only if, for every layout this document defines, it produces exactly the specified bytes when encoding and accepts exactly the specified language of byte strings when decoding.

Decode discipline is **reject, never normalize** — the workspace-wide convention the core specification establishes in Appendix D (canonical serialization determinism and ordered iteration). Concretely, each of the following is a *decode error*, never an input to be repaired:

- an unknown discriminant in any tagged union;
- a set- or map-valued field whose elements are not in strictly ascending canonical order, or which contains duplicates;
- text that is not in Unicode NFC where NFC is required (catalog identifiers, barrier pitch-space names, operation labels);
- trailing bytes after a complete top-level value (Requirement 2.2);
- recursion beyond a normative depth bound (Requirement 8.3).

A decoder **MUST NOT** silently sort, deduplicate, NFC-fold, truncate, or otherwise canonicalize non-canonical input: two implementations that disagree on repair would disagree on canonical state.

The **golden anchor registry** (Chapter 11) is the conformance contract binding this text to the reference implementation: every layout in this document is anchored by at least one named test in the reference crates, and a change that breaks an anchor is either a defect or a deliberate, versioned format revision (Chapter 9).

Chapter 2

Encoding Conventions

This chapter restates the ratified convention baseline (core specification, requirement `req:format:codec-conventions`) as normative fact and completes it: the three prefix/endianness regimes, the primitive composition rules, the varint disposition, and the trailing-bytes rule.

2.1 The Three Encoding Regimes

Three internally consistent regimes coexist in the format. Every layout in this document belongs to exactly one of them, and each layout's chapter says which.

Regime	Surfaces	Rules
(a)	Canonical document encodings: the core value codec (Chapter 5), the operation layer (Chapter 6), the bundle codec (Chapter 7).	Little-endian fixed-width integers; `u32` little-endian counts and length prefixes on every variable-width field.
(b)	Layout-side pinned encodings: the extension-declaration blobs and edit-barrier trees (Chapter 8) and the `ResolvedLayoutIR` canonical output (Chapter 10).	Little-endian integers; `u64` little-endian counts and length prefixes — a golden-locked divergence from regime (a). Schema major 1 moves the *resolved-layout* surface to `u32`; the manifest-embedded barrier blobs stay `u64` (Section 9.4).
(c)	Identifiers and hash-derived forms: the 128-bit identifier family, `ReplicaId`, `OperationId`, `ConflictId`, the `TypedObjectId` discriminant, digest truncations (Chapter 3).	Big-endian fixed width. Numeric order equals canonical byte order, which is what Appendix D's ordered iteration requires.

Rationale

In schema major 0, regimes (a) and (b) differ only in prefix width, and the divergence was *tolerated rather than repaired*: the two surfaces were golden-locked independently (the barrier blobs and the resolved-layout output carry their own literal-byte anchors), they

never embed one another's framing, and unifying the prefix width within major 0 would break locked bytes for zero benefit. Once a major bump was scheduled for data-model reasons, part of the unification became free to carry: schema major 1 moves the resolved-layout surface (an independent non-canonical `LayoutCache`) to regime (a)'s `u32` prefixes and re-locks its goldens. The barrier blobs cannot follow in the same step — they are carried opaquely inside the *canonical* manifest, which stays major 0 — so they keep regime (b) until a manifest- revising major (Chapter 9). Regime (c) is not a divergence at all: an identifier's byte form *is* its sort key, so big-endian is load-bearing.

2.2 Primitive Composition Rules

The following rules apply throughout regime (a); regimes (b) and (c) state their deltas in their own chapters.

Booleans. A single byte: `0` = false, `1` = true. Any other value is a decode error.

Integers. Fixed-width little-endian two's complement (`u8`/`i8` through `u128`), except within regime (c).

`Option<T>`. A presence byte: `0` = absent (no further bytes), `1` = present, followed by `T`'s encoding. A presence byte greater than 1 is a decode error.

Sequences, sets, maps. A `u32` little-endian element count, then each element (for maps: key then value). Set- and map-valued fields are emitted in canonical iteration order — ascending by the key's canonical byte form — and decoders reject out-of-order or duplicated elements wherever the field is a set or map (as opposed to an order-significant list).

Free text. A `u32` little-endian byte length, then the raw UTF-8 bytes. Free-text fields (e.g. a score title) are **never** NFC-folded by the codec: $\text{decode}(\text{encode}(x)) = x$ is the round-trip identity, and folding would break it. By contrast, *catalog identifiers* (pitch-space ids, tuning-system ids, and the other `catalog_id` newtypes) are NFC-normalized *at construction*, so their stored form is already NFC; the codec encodes the stored string verbatim. Operation-layer text fields (the transaction label, conflict field paths) are NFC-normalized at encode time (Chapter 6).

Structs. Positional, unframed concatenation of the fields' encodings in declaration order. No field tags, no per-field lengths, no padding, no alignment. (This is the layout the frozen-layout rule, Requirement 5.3, freezes.)

Tagged unions. A single discriminant byte, then the selected variant's payload encoded positionally. Two documented exceptions: `TypedObjectId` uses a *16-bit big-endian* discriminant (core requirement `req:graph:typed-object-id-discriminants`; Chapter 3), and the bundle's `ProfileId` uses a `u32` little-endian discriminant (core requirement `req:format:profileid-discriminants`; Chapter 7).

Determinism-canonical leaves. Where a composite codec embeds a value whose canonical byte form is owned by the determinism layer or the identifier layer (a graph identifier, `ContentHash`, `CanonicalF64`, `RationalTime`, a wall-clock integer), the core value codec frames it as a `u32` little-endian length prefix followed by the leaf's own canonical bytes

— even when the leaf is fixed-width. The leaf's bytes are then self-describing under its own decoder. (The operation layer instead embeds fixed-width leaves *raw*, unprefixed; Chapter 6 states this explicitly.)

REQUIREMENT 2.1

No varint. Schema major 0 defines *no* variable-length integer encoding. Every integer field in every layout in this document is fixed-width. The core specification's "varint conventions" language (Chapter 8, BINARY FORMAT COMPANION) is hereby discharged as *none*: an encoder **MUST NOT** emit, and a decoder **MUST NOT** accept, any LEB128-style or otherwise variable-width integer under schema major 0. Variable width in this format comes only from explicit `u32`/`u64` length prefixes and from `RationalTime`'s length-prefixed big-integer magnitudes (Chapter 4).

REQUIREMENT 2.2

Trailing bytes. Every top-level decode entry point — a whole-`Score` value, a per-value `CanonicalValue` decode, an `OperationKindTag`, a `MaterializedState`, a manifest payload, an operation-index payload, a block payload, an extension-declaration blob, a fixed-width primitive — **MUST** reject input with bytes remaining after the complete value has been decoded. Trailing bytes are a decode error, never ignored padding.

Chapter 3

Identifiers and Derivations

This chapter is an *import-by-citation* of the core specification's Canonical Byte-Layout Reference appendix (`app:bytes`): that appendix and its golden anchors govern every table in this chapter, and the tables here are compact restatements for self-containedness, not a second source of truth. All layouts in this chapter are regime (c): big-endian, fixed width.

3.1 The 128-Bit Identifier Family

`ReplicaId`. 8 bytes, big-endian `u64`. The reserved system namespace is `ReplicaId::SYSTEM_DERIVED` = `0xFFFF_FFFF_FFFF_FFFF`; user-authored replicas **MUST NOT** use it.

Typed graph identifiers. Every typed 128-bit graph identifier (`EventId`, `PitchId`, `VoiceId`, `RegionId`, `StaffInstanceId`, `TransactionId`, … — the `graph_id` family) packs the 64-bit replica into the high half and the counter into the low half, and encodes as exactly 16 bytes: replica big-endian (8) ‖ counter big-endian (8). Numeric order, canonical byte order, and Appendix-D iteration order coincide.

`OperationId`. The same 16-byte form: replica big-endian (8) ‖ counter big-endian (8). Ordering is (*replica*, *counter*) lexicographic, which equals byte order.

Registry and author identifiers (operation layer). The opaque 128-bit registry newtypes (`OperationKindRegistryId`, `ConflictKindRegistryId`, `ResolutionRegistryId`, `RepairKindRegistryId`, `ReanchorReasonRegistryId`, `ReplicaAnomalyRegistryId`, `IntegrityAnomalyRegistryId`, `ExtensionPreconditionId`, `PreconditionFailureRegistryId`) and `AuthorId` all encode as 16 big-endian bytes.

3.2 `TypedObjectId`

Canonical form: a **16-bit big-endian discriminant** ‖ the variant payload (core requirement `req:graph:typed-object-id-discriminants`). Non-`Registered` variants carry one 16-byte identifier, for a total of **18 bytes**; `Registered` carries the `ObjectKindRegistryId` (16 big-endian bytes) ‖ the extension's own raw `u128` (16 big-endian bytes), for a total of **34 bytes**. An unknown discriminant is a decode error.

Disc	Variant	Disc	Variant
0	`Event`	14	`Spanner`
1	`Pitch`	15	`Marker`

Disc	Variant	Disc	Variant
2	`Voice`	16	`AnalyticalAnnotation`
3	`Staff`	17	`Comment`
4	`StaffInstance`	18	`GraphicObject`
5	`StaffGroup`	19	`GraphicGesture`
6	`Region`	20	`TimeSignature`
7	`Instrument`	21	`AnalysisLayer`
8	`PartDefinition`	22	`Tuplet`
9	`Measure`	23	`RepeatStructure`
10	`BarlineAlignmentGroup`	24	`LyricLine`
11	`Slur`	25	`ChordSymbol`
12	`Tie`	26	`View`
13	`Beam`	27	`Registered`

3.3 System-Derived Counters and Digest Truncations

All content derivation uses BLAKE3-256 over a domain-separated preimage: the 8-byte domain tag is always the first bytes hashed. Two truncations recur, both taking the digest's *leading* bytes big-endian:

- $\text{trunc64}(d) = \texttt{u64} :: \texttt{from_be_bytes}(d[0..8])$ — the counter of a system-derived identifier;
- $\text{trunc128}(d) = \texttt{u128} :: \texttt{from_be_bytes}(d[0..16])$ — content-derived 128-bit identifiers.

The system-derived counter function is $\text{trunc64}(\text{BLAKE3}(\textit{tag} \,\|\, \textit{inputs}))$; the resulting identifier is `from_parts(ReplicaId::SYSTEM_DERIVED,` *counter*`)`. Only a system domain tag (prefix `MUSCS`) may seed it. The ratified instances (governed by `app:bytes` and the cited requirements):

Identifier	Tag	Canonical inputs
Promoted `VoiceId`	`MUSCSVCE`	64-byte preimage: `staff_instance` (16 BE) ‖ `original_voice` (16 BE) ‖ `winning_op` (16 BE) ‖ `losing_op` (16 BE). Core System-Promoted Voices.
System `PitchId`	`MUSCSPCH`	The pitch's intrinsic canonical bytes (core requirement `req:graph:system-derived-pitch-id`).
`IntegrityAnomalyId`	`MUSCSANM`	The anomaly kind's canonical bytes (Chapter 6; core requirement `req:graph:integrity-anomaly-id`).

The 128-bit content-derived identifiers:

Identifier	Tag	Preimage after the tag
`ConflictId`	`MUSCCONF`	`kind.canonical_bytes()` ‖ causing operations (each 16 BE, sorted by canonical bytes) ‖ affected objects (each `TypedObjectId` canonical bytes, sorted); trunc128. Wire form of the id itself: 16 big-endian bytes.

Identifier	Tag	Preimage after the tag
`ManifestId`	`MUSCMNIF`	`document_id` (16) ‖ `generation` (`u64` LE) ‖ manifest body bytes (body excludes the `manifest_id` field); trunc128. Restated normatively in Chapter 7 (core requirement `req:format:manifest-id`). Note the *on-disk* field encodes little-endian (Chapter 7).
`EnvelopeHash`	`MUSCENVH`	The envelope's complete canonical bytes; the full 32-byte digest (no truncation). Chapter 6.
`LayoutObjectId`	`MUSCLOID`	Non-canonical; three preimage shapes, Chapter 10 (core requirement `req:layoutir:object-id-derivation`).
`BlobId`	`MUSCBLOB`	The blob payload, bare (`req:format:blob-hash-shape`); full 32-byte digest. Chapter 4.

3.4 The Domain-Tag Registry

Every domain tag is exactly 8 printable-ASCII bytes beginning `MUSC`; the vocabulary is closed (governed by `app:bytes`, DOMAIN-TAG REGISTRY). Extension system tags **MUST** begin `MUSCS`, be exactly 8 bytes, and collide with neither the built-ins nor the file magics.

Tag	Name	Use
`MUSCCHNK`	chunk	non-manifest chunk hash preimages
`MUSCMANI`	manifest	manifest chunk hash preimages
`MUSCBLOB`	blob	`BlobId` (bare *tag* ‖ *payload*)
`MUSCCONF`	conflict	`ConflictId` derivation
`MUSCENVH`	envelope	`EnvelopeHash`
`MUSCFNTM`	font metrics	glyph-catalog metrics hash (non-canonical)
`MUSCMNIF`	manifest id	`ManifestId` derivation
`MUSCSVCE`	system voice	promoted-voice counters
`MUSCSPCH`	system pitch	system-derived pitch counters
`MUSCSANM`	system anomaly	`IntegrityAnomalyId`
`MUSCLOID`	layout object id	`LayoutObjectId` (non-canonical)
`MUSCBND\0`	bundle magic	file magic, *not* a hash tag (note the trailing NUL)
`MUSCSUPR`	superblock magic	slot magic, *not* a hash tag

3.5 `SnapshotId`

The core specification's support-type identity table says *"the Binary Format companion defines the exact derivation"* of `SnapshotId`. vo has no snapshot producer — pruning and canonical-base creation are deferred — so there is nothing to derive *from* yet.

REQUIREMENT 3.1

In schema major 0, `SnapshotId` is an **opaque 16-byte identifier**: 16 raw bytes on the wire (Chapter 4), with no normative derivation function. Readers **MUST** treat two `SnapshotId`s as equal iff their 16 bytes are equal and **MUST NOT** attempt to derive or verify a snapshot id from snapshot content.

Open Question

The `SnapshotId` derivation (presumably a content derivation in the `trunc128` family, over the snapshot's canonical payload and its covering frontier) is deferred together with the pruning/garbage-collection track that will produce the first snapshots. It must be pinned before any writer emits a `canonical_base`; until then the opaque reading above is complete.

Chapter 4

Primitive Value Encodings

The leaf encodings owned by the determinism layer and the small fixed identifier types. These byte forms are what the framings of Chapters 5–7 embed.

Type	Width	Encoding
`CanonicalF64`	8	IEEE-754 binary64, little-endian. -0.0 is canonicalized to $+0.0$ at construction, so the negative-zero bit pattern never appears in canonical bytes. Decode rejects NaN and $\pm\infty$ (non-finite floats are corruption).
`QuantizedCoord`	8	`i64` little-endian, in units of 1/1024 staff space (`STAFF_SPACE_GRID` = 1024). Construction from a float rounds ties-to-even and rejects non-finite or out-of-range input.
`ContentHash`	32	Raw BLAKE3-256 digest bytes.
`ChunkId`	32	Raw digest bytes (a `ContentHash` newtype).
`DomainTag`	8	The raw ASCII tag bytes. Decode validates against the closed vocabulary of Section 3.4; foreign bytes are a decode error.
`WallClockTime`	8	`i64` little-endian, nanoseconds. Floating-point wall-clock time is forbidden in stored data.
`WallClockDuration`	8	`i64` little-endian, nanoseconds.
`SchemaVersion`	4	`major` `u16` LE ‖ `minor` `u16` LE.
`SemVer`	12	`major` `u32` LE ‖ `minor` `u32` LE ‖ `patch` `u32` LE. Canonical *ordering* of `SemVer`-keyed collections is **numeric** on (*major*, *minor*, *patch*), never byte-lexicographic on the little-endian encoding (which would order 256.0.0 before 1.0.0).
`ReductionAlgorithmVersion`	4	`u32` little-endian.
`FileUuid`, `DocumentId`, `LineageId`, `SnapshotId`, `ExtensionId`, `ProfileRegistryId`	16	16 raw opaque bytes each. No internal structure is interpreted; equality is byte equality.
`BlobId`	32	Raw digest bytes: BLAKE3(`MUSCBLOB` ‖ *payload*), the only bare *tag* ‖ *payload* content hash in the format (core requirement `req:format:blob-hash-shape`).
`FrontierBytes`	var	`u32` LE length ‖ opaque bytes (an ops-computed causal frontier the bundle carries verbatim).

4.1 `RationalTime`

Restating core requirement `req:format:rationaltime-encoding`, which governs. `RationalTime` is an arbitrary-precision rational, always stored *reduced* (equal rationals encode identically; the sign lives on the numerator; the denominator is strictly positive). Its canonical byte form:

Field	Encoding
sign	1 byte: 0 = zero, 1 = positive, 2 = negative
numerator	`u32` LE magnitude byte length ‖ big-endian magnitude bytes
denominator	`u32` LE magnitude byte length ‖ big-endian magnitude bytes

Decode rejects a sign byte greater than 2, a zero denominator, and trailing bytes. `MusicalPosition` and `MusicalDuration` are newtypes whose canonical bytes are exactly their inner `RationalTime`'s. Wall-clock integers deliberately do *not* use this form: they are the fixed-width little-endian integers of the table above, matching `QuantizedCoord`'s convention.

Chapter 5

Graph Value Layouts

This chapter defines the schema-major-0 wire form of every value reachable from a `Score` — the whole-document composite codec. The core specification's `DECISIONS` record (P11-4) held this codec *provisional* pending this companion; Requirement 5.1 and Requirement 5.3 **ratify** it: the reference implementation's layout, as specified here, *is* the schema-major-0 wire form, and P11-4's provisional status is discharged.

All layouts in this chapter are regime (a): little-endian, `u32` prefixes.

5.1 The `Score` Envelope

Requirement 5.1

The canonical bytes of a `Score` are the bare positional concatenation of its nineteen fields' encodings, in exactly this order:

1. `metadata`
2. `canvas`
3. `instruments`
4. `staves`
5. `staff_groups`
6. `parts`
7. `cross_cutting`
8. `time_signatures`
9. `tuning_context`
10. `tempo_map`
11. `events`
12. `spelling_attachments`
13. `decomposition_attachments`
14. `spelling_precedence`
15. `analysis_layers`
16. `views`

17. `identity`
18. `tombstoned_pitches`
19. `tombstoned_events`

There is **no magic** and **no version prefix** at the value level: format identification lives in the bundle's fixed header (Chapter 7) and versioning lives at the chunk layer (`SchemaVersion` in every `ChunkRef` and superblock; Chapter 9). Decoders **MUST** reject trailing bytes after the nineteenth field.

5.2 Composition Rules as Normative Encoding Facts

The reference codec is built from a small set of conventions; each is a normative statement about the bytes, not an implementation detail:

Positional structs. Every struct in this chapter encodes as its fields in declaration order, unframed (Section 2.2). Zero-field structs encode as zero bytes.

C-style enums. A fieldless enum is a single discriminant byte (tables in Section 5.3).

Tagged unions. A discriminant byte, then the variant's fields positionally.

Leaf framing. The determinism-canonical leaf types — `ReplicaId`, `OperationId`, every embedded typed graph identifier (`EventId`, `PitchId`, `VoiceId`, `StaffId`, `StaffInstanceId`, `StaffGroupId`, `RegionId`, `InstrumentId`, `PartDefinitionId`, `MeasureId`, `BarlineAlignmentGroupId`, `SlurId`, `TieId`, `BeamId`, `SpannerId`, `TupletId`, `MarkerId`, `AnalyticalAnnotationId`, `CommentId`, `RepeatStructureId`, `LyricLineId`, `ChordSymbolId`, `GraphicObjectId`, `GraphicGestureId`, `TimeSignatureId`, `AnalysisLayerId`, `ViewId`) — plus `ContentHash`, `CanonicalF64`, `RationalTime`, `MusicalPosition`, `MusicalDuration`, `WallClockTime`, and `WallClockDuration`, are each emitted as `u32` LE length ‖ the leaf's canonical bytes (Chapters 3 and 4). An embedded 16-byte identifier therefore occupies 20 bytes inside a `Score`. Decode runs the leaf's own validating decoder over the framed region.

Catalog identifiers. The `catalog_id` newtypes (`PitchSpaceId`, `TuningSystemId`, `AccidentalId`, registry-name ids, ...) encode as their stored NFC string under the String rule (`u32` LE length ‖ UTF-8).

`EventArena`. `u32` LE event count, then each `Event` in ascending-`EventId` order (identity travels inside each event; the arena adds no per-event framing).

Validated reconstruction. Types with checked constructors decode through them: `TupletRatio` rejects degenerate ratios, `KeySignature` encodes its `fifths()` as `i8` and rejects out-of-range values, `TimeSignature` re-validates that beat groups sum to the measure duration, `NonZeroU16` rejects zero, `EventOrderingDAG` rejects cyclic edge maps, `ReferencePitch`, `Tempo`, and `SpellingPrecedence` re-validate their invariants. Structurally well-formed bytes that fail a type invariant are a decode error (reject-never-normalize).

5.3 Discriminant Tables

The complete discriminant assignment for schema major 0. Every table is governed by Requirement 5.3: an unknown discriminant is a decode error; changes are gated by Chapter 9.

5.3.1 C-style enums (one byte, no payload)

Type	Discriminants
`MeasurePosition`	0 Start, 1 End
`RegionEdge`	0 Start, 1 End
`CmnNominal`	0 C, 1 D, 2 E, 3 F, 4 G, 5 A, 6 B
`SpellingSourceKind`	0 UserChosen, 1 Imported, 2 Propagated, 3 Inferred, 4 Analytical
`TempoShape`	0 Constant, 1 Linear, 2 Exponential, 3 Curve
`StemDirection`	0 Up, 1 Down
`MeasureNumberVisibility`	0 Auto, 1 Always, 2 Never
`Aleatoric`	0 Musical, 1 WallClock, 2 EitherPerEvent, 3 FreelyMixed
`AnchoringDiscipline`	
`NoteValue`	0 Whole, 1 Half, 2 Quarter, 3 Eighth, 4 Sixteenth, 5 ThirtySecond, 6 SixtyFourth
`ClefShape`	0 G, 1 F, 2 C, 3 Percussion

5.3.2 Tagged unions (one byte, then the variant payload)

Type	Discriminants
`AnchorOffset`	0 Musical, 1 WallClock, 2 Zero
`TimeAnchor`	0 Event, 1 Measure, 2 Region, 3 WallClock (Section 5.4)
`EventPosition`	0 Musical, 1 WallClock
`ConcreteDuration`	0 Musical, 1 WallClock
`EventDuration`	0 Musical, 1 WallClock, 2 Indeterminate
`TimeBounds`	0 MusicalRange, 1 WallClockRange, 2 Unbounded
`PitchSpacePosition`	0 Cmn, 1 Integer, 2 JiVector, 3 Registered
`TuningReference`	0 Inherit, 1 Explicit
`AcousticRealization`	0 Implicit, 1 CentsOffset, 2 AbsoluteHz
`SpellingNominal`	0 Cmn, 1 Integer, 2 Registered
`SpellingSource`	0 UserChosen, 1 Inferred, 2 Imported, 3 Propagated, 4 Analytical
`SpellingScope`	0 Pitch, 1 Range
`SpellingDirective`	0 Explicit, 1 Rule
`VoiceSelector`	0 All, 1 Voices
`GraceKind`	0 Acciaccatura, 1 Appoggiatura, 2 Unmeasured, 3 MeasuredFraction
`IndeterminacyKind`	0 Pitch, 1 Duration, 2 Choice, 3 Compound
`TrajectoryEndpoint`	0 EventPitch, 1 ExplicitPitch
`TrajectoryShape`	0 Linear, 1 Exponential, 2 Curve, 3 Stepwise
`Event`	0 Pitched, 1 Unpitched, 2 Rest, 3 Indeterminate, 4 Trajectory, 5 Graphic, 6 Cue (Section 5.4)
`RegionTimeModel`	0 Metric, 1 Proportional, 2 Aleatoric
`RegionContent`	0 StaffBased, 1 FreeGraphic, 2 Hybrid

Type	Discriminants
`VoiceOrigin`	0 UserDeclared, 1 Imported, 2 SystemPromoted
`StaffGroupKind`	0 GrandStaff, 1 Bracket, 2 SubBracket, 3 Choral, 4 Registered
`TieClass`	0 Standard, 1 Editorial, 2 CrossVoice, 3 LaissezVibrer, 4 Registered
`AnnotationAnchor`	0 Event, 1 Range, 2 Region
`GestureAnchoring`	0 Events, 1 Range, 2 Free
`DecompositionSource`	0 UserChosen, 1 Inferred, 2 Imported, 3 Propagated
`TimeSignatureDisplay`	0 Standard, 1 Compound, 2 Irrational, 3 MixedDenominators, 4 None, 5 Symbolic

REQUIREMENT 5.2

The `SpellingSource` / `SpellingSourceKind` trap. These two vocabularies name the same five provenance concepts but assign them *different* discriminants: `SpellingSourceKind` (the C-style precedence-list element) is 0 UserChosen, 1 Imported, 2 Propagated, 3 Inferred, 4 Analytical, while `SpellingSource` (the payload-carrying attachment provenance) is 0 UserChosen, 1 Inferred, 2 Imported, 3 Propagated, 4 Analytical. The orders differ (`Inferred` is 3 in one and 1 in the other), both are golden-locked, and an implementation **MUST NOT** share one discriminant function between them.

5.4 Representative Complete Layouts

The following layouts are spelled byte-by-byte both as worked examples of the composition rules and as normative fact. Recall that inside this chapter's codec every leaf is framed (`u32` LE length ‖ bytes), so an embedded 16-byte identifier occupies $4 + 16 = 20$ bytes.

5.4.1 `RationalTime`

As a leaf inside a composite: `u32` LE length ‖ the canonical form of Section 4.1 (sign ‖ length-prefixed big-endian numerator magnitude ‖ length-prefixed big-endian denominator magnitude).

5.4.2 `TimeAnchor`

One discriminant byte, then:

0 Event `id` (`EventId` leaf, 20) ‖ `offset` (`AnchorOffset`: tag byte, then for Musical a `MusicalDuration` leaf, for WallClock a `WallClockDuration` leaf ($4 + 8 = 12$), for Zero nothing).

1 Measure `id` (`MeasureId` leaf, 20) ‖ `position` (`MeasurePosition`, 1) ‖ `offset` (`AnchorOffset`).

2 Region `id` (`RegionId` leaf, 20) ‖ `edge` (`RegionEdge`, 1) ‖ `offset` (`AnchorOffset`).

3 WallClock `time` (`WallClockTime` leaf, 12).

5.4.3 `Pitch`

Positional struct: `scale_position` ‖ `acoustic`.

- `ScalePosition` = `space` (`PitchSpaceId`: NFC string, `u32` LE length ‖ UTF-8) ‖ `position` (`PitchSpacePosition`: tag byte, then Cmn = `nominal` (`CmnNominal`, 1) ‖ `alteration` (`i8`, 1) ‖ `octave` (`i8`, 1); Integer = `space_size` (`u16` LE, 2) ‖ `index` (`i32` LE, 4); JiVector = `u32` LE count ‖ that many `i32` LE components; Registered = a catalog-id string).
- `AcousticPitch` = `tuning` (`TuningReference`: tag byte; Explicit carries a `TuningSystemId` catalog-id string) ‖ `realization` (`AcousticRealization`: tag byte; CentsOffset and AbsoluteHz each carry a `CanonicalF64` leaf, $4 + 8 = 12$).

5.4.4 `Event`

One discriminant byte (0–6), then the variant struct positionally. All seven variants share the prefix `id` (`EventId` leaf) ‖ `voice` (`VoiceId` leaf) ‖ `position` (`EventPosition`) ‖ `duration` (`EventDuration`); the remaining fields, in order:

Disc	Variant	Fields after the common prefix
0	`Pitched`	`pitches` ‖ `articulations` ‖ `dynamic` ‖ `ornaments` ‖ `stem` ‖ `grace`
1	`Unpitched`	`staff_position` ‖ `instrument_member` ‖ `articulations` ‖ `dynamic` ‖ `stem` ‖ `grace`
2	`Rest`	`vertical_position` ‖ `visible`
3	`Indeterminate`	`indeterminacy` ‖ `hints`
4	`Trajectory`	`start` ‖ `end` ‖ `shape` ‖ `display`
5	`Graphic`	`graphics` ‖ `playback_bindings`
6	`Cue`	`source` ‖ `rendering`

Each field encodes under this chapter's rules for its type (vectors are `u32`-counted, options carry a presence byte, embedded identifiers are leaves, unions carry their tag byte).

5.4.5 `Slur`

Positional struct of three leaves: `id` (`SlurId` leaf, 20) ‖ `start_event` (`EventId` leaf, 20) ‖ `end_event` (`EventId` leaf, 20) — 60 bytes total.

5.5 The `CanonicalValue` Seam

The public per-value codec — the seam between this document and the Operation Catalog's value-typed payloads (Operation Catalog, VALUE-TYPED PAYLOADS, requirement `req:catalog:value-encoding`) — is defined for exactly these forty-one types, in the order the core codec declares them (the Phase-3 operation tranche added `Staff`/`TimeSignature`/`TempoSegment`/`StaffLineConfiguration`; the repeat-authoring revision `RepeatStructure`; the genesis tranches the settings and entity values their operations carry; X3.1 `Tuplet`; and X3.6 `Clef` and `KeySignature`):

`Event`, `Rest`, `Pitch`, `IdentifiedPitch`, `PitchSpelling`, `Tie`, `Slur`, `Beam`, `Spanner`, `RegionTimeModel`, `TimeAnchor`, `Region`, `StaffInstance`, `Voice`, `DecompositionAttachment`, `SpellingSourceKind`, `ScoreMetadata`, `MetricGrid`, `Staff`, `TimeSignature`, `TempoSegment`, `StaffLineConfiguration`, `Instrument`, `RepeatStructure`, `RationalTime`, `Clef`, `KeySignature`, `ScoreTuningContext`, `TuningOverride`, `TuningScope`, `SmuflVersionRequirement`, `SmuflVersion`, `CanvasLayoutDefaults`, `SpellingPrecedence`, `TuningContextSettings`, `StaffGroup`, `PartDefinition`, `AnalysisLayer`, `ViewDefinition`, `Measure`, `Tuplet`.

The seam introduces **no new bytes**: a value's stand-alone canonical bytes are byte-for-byte the bytes the whole-`Score` codec embeds for that value. Stand-alone decode applies the same validation and rejects trailing bytes.

5.6 The Frozen-Layout Rule

REQUIREMENT 5.3

Frozen positional layouts (the schema-evolution keystone). Within schema major 0, every positional struct layout in this chapter is **frozen**: there is no field-addition mechanism at the value level — no per-field framing, no optional-field encoding, no trailing-field tolerance. Any change to any struct's field *set* or field *order* is a schema-MAJOR change and requires a documented migration in this document's revision history (Chapter 9). Appending a *new variant* to an **open** discriminant vocabulary is a schema-MINOR change under the rules of Chapter 9. At the value layer, the open vocabularies are those with a `Registered` escape variant — `PitchSpacePosition`, `SpellingNominal`, `StaffGroupKind`, `TieClass` — through which extensions attach without any wire change at all; every *other* union in Section 5.3 (including `Event`, `TimeAnchor`, `TimeSignatureDisplay`) is closed in v0 and may grow only by an append ratified in a revision of this document.
This rule ratifies the project's staging decision that data-model payload expansion — `SlurKind`, beam geometry, voltas, instrument bodies, score-metadata growth, and their kin — lands as a coordinated schema-MAJOR revision of this document, not as ad-hoc field insertion.

Chapter 6

Operation Wire Forms

This chapter pins the operation layer: envelopes, payloads, stamps, causal contexts, effects, and materialized state. It **ratifies** the encodings the ops crate's `DECISIONS` record held as "provisional canonical encoding" (mirroring core P11-4); that provisional status is discharged.

Regime (a) applies, with one composition difference from Chapter 5: the operation layer embeds *fixed-width* canonical leaves (identifiers, hashes, `ConflictId`) **raw**, without a length prefix (`push_canon`); only variable-width parts carry `u32` LE length prefixes (`push_lp_bytes`). Sequences (`push_seq`) are a `u32` LE count, then each element individually length-prefixed (`u32` LE), regardless of element width. Text (`push_str`) is NFC-normalized at encode time, then length-prefixed UTF-8. Embedded graph values are the `CanonicalValue` bytes of Section 5.5, framed by a `u32` LE length prefix (Operation Catalog requirement `req:catalog:value-encoding`).

6.1 The Operation Envelope

Requirement 6.1

An `OperationEnvelope`'s canonical bytes are, in order:

Field	Encoding
`id`	`OperationId`, 16 big-endian bytes, raw
`author`	`AuthorId`, 16 big-endian bytes, raw
`stamp`	`OperationStamp`, 28 bytes (Section 6.5)
`causal_context`	`CausalContext` (Section 6.5)
`transaction`	option tag byte (0 absent / 1 present), then `TransactionId` 16 BE if present
`payload`	`OperationPayload` (Section 6.2)

Id-leads property. The envelope's first 16 bytes *are* the `OperationId`'s canonical bytes. This is load-bearing: the operation index (Chapter 7, Section 7.6) keys its entries on those leading 16 bytes without decoding envelopes, and the sanctioned partial read (*peek*) of an envelope is exactly its leading 16 bytes. Any future envelope revision **MUST** preserve id-leads or revise the operation index in the same schema-major step.

`EnvelopeHash` = BLAKE3(`MUSCENVH` ‖ *envelope canonical bytes*) — the full 32-byte digest.

Envelopes are **encode-only** at this layer, deliberately: once committed, an envelope is stored and transported as opaque bytes and is never reconstructed into typed form by the storage layer; the peek above is the only sanctioned partial read. (Reducers decode payloads they authored or received through the ops layer's own typed surface; the wire contract is that stored envelope bytes are preserved verbatim, since `EnvelopeHash` commits to them.)

6.2 `OperationPayload` and the Meta-Operations

`OperationPayload`: one discriminant byte, then the variant.

Disc	Variant	Payload
0	`Primitive`	an `OperationKind` (Section 6.3)
1	`ResolveConflict`	`target` (`ConflictId`, 16 BE) ‖ `action` (`ResolutionAction`)
2	`UndoTransaction`	`target` (`TransactionId`, 16 BE) ‖ `policy` (`UndoPolicy`, 1 byte)
3	`ResolveEquivocation`	`target` (`OperationId`, 16 BE) ‖ `chosen` (`EnvelopeHash`, 32)

Discriminants 0–2 are the ratified v1 values; 3 was appended by the catalog's RESOLVEEQUIVOCATION (META-OPERATION) entry. The vocabulary is append-only: new meta-operations take ≥ 4.

`UndoPolicy` is one byte: 0 StrictInverse, 1 BestEffort, 2 Cascade. `ResolutionAction` is one byte then an optional payload (`app:bytes`, core requirement `req:semops:resolution-action-discriminants`): 0 AcceptLoser, 1 KeepWinner, 2 Override (‖ `OperationId` 16 BE), 3 Reanchor (‖ `TypedObjectId`), 4 Dismiss, 5 Registered (‖ `ResolutionRegistryId` 16 BE).

6.3 `OperationKind`: the Primitive Wire Discriminants

REQUIREMENT 6.2

The `OperationKind` wire discriminant is one byte, golden-locked to the table below. The vocabulary is **append-only**: new primitive kinds take discriminants past 29; the assignments below never change.

Each row also pins the payload's byte layout; the field *set and order* is the Operation Catalog's (the cited section governs semantics). "lp(T)" means the `u32`-LE-length-prefixed `CanonicalValue` bytes of a `T` (Section 5.5); bare identifiers are raw 16 big-endian bytes; "seq$^{\uparrow}$" is a `push_seq` sequence sorted ascending by canonical bytes, and "seq$^{\Uparrow}$" is the same sequence constrained to be *strictly* increasing — a set, not a multiset. A decoder **MUST** reject a seq$^{\Uparrow}$ containing a duplicate rather than normalize it away, since the duplicate is the only evidence that the writer and the reader disagree about the field's type.

Disc	Kind	Payload layout	Catalog section
0	`InsertEvent`	`staff_instance` (16) ‖ lp(`Event`)	INSERTEVENT

Disc	Kind	Payload layout	Catalog section
1	`DeleteEvent`	`event` (16) ‖ `TupletCompensation`	DeleteEvent
2	`RespellPitch`	`pitch` (16) ‖ lp(`PitchSpelling`)	RespellPitch
3	`CreateCrossCutting`	`CrossCuttingValue`	CreateCrossCutting
4	`ChangeRegionTimeModel`	`region` (16) ‖ lp(`RegionTimeModel`) ‖ seq↑(`EventId`) ‖ `PositionRemapping`	ChangeRegion TimeModel
5	`SetUserSystemBreak`	`region` (16) ‖ lp(`TimeAnchor`) ‖ `present` (bool, 1)	SetUserSystemBreak
6	`DeclareTransaction`	`id` (16) ‖ `label` (NFC string) ‖ option tag ‖ `TransactionCategory` if present	DeclareTransaction
7	`Registered`	`OperationKindRegistryId` (16) ‖ lp(opaque bytes)	K1 — Framework Slots
8	`ModifyEvent`	lp(`Event`)	ModifyEvent
9	`Transpose`	seq↑(`PitchId`) ‖ `chromatic_steps` (i32 LE, 4). Frozen: the sequence is a multiset, *not* seq⇑	Transpose
10	`InsertIdentifiedPitch`	`event` (16) ‖ lp(`IdentifiedPitch`)	Identified-Pitch Operations
11	`DeleteIdentifiedPitch`	`pitch` (16)	Identified-Pitch Operations
12	`ModifyIdentifiedPitch`	`pitch` (16) ‖ lp(`Pitch`)	Identified-Pitch Operations
13	`DeleteCrossCutting`	`structure` (`TypedObjectId`, 18/34)	DeleteCrossCutting
14	`ModifyCrossCutting`	`CrossCuttingValue`	ModifyCrossCutting
15	`CreateRegion`	lp(`Region`)	Structural Containers
16	`DeleteRegion`	`region` (16)	Structural Containers
17	`CreateStaffInstance`	`region` (16) ‖ lp(`StaffInstance`)	Structural Containers
18	`DeleteStaffInstance`	`staff_instance` (16)	Structural Containers
19	`CreateVoice`	`staff_instance` (16) ‖ lp(`Voice`)	Structural Containers
20	`DeleteVoice`	`voice` (16)	Structural Containers
21	`SetMetadata`	lp(`ScoreMetadata`)	Score Settings
22	`SetMetricGrid`	`region` (16) ‖ option tag (0/1) ‖ lp(`MetricGrid`) if present	Score Settings
23	`SetUserPageBreak`	`region` (16) ‖ lp(`TimeAnchor`) ‖ `present` (bool, 1)	Score Settings
24	`CreateStaff`	lp(`Staff`)	CreateStaff
25	`SetTimeSignature`	`region` (16) ‖ lp(`TimeAnchor`) ‖ option tag (0/1) ‖ lp(`TimeSignature`) if present	Meter and Tempo Overwrites
26	`SetTempoSegment`	option tag ‖ `region` (16) if present ‖ lp(`TimeAnchor`) ‖ option tag ‖ lp(`TempoSegment`) if present	Meter and Tempo Overwrites
27	`SetStaffLayout`	`staff_instance` (16) ‖ option tag ‖ `instrument_override` (16) if present ‖ option tag ‖ lp(`StaffLineConfiguration`) if present ‖ `visible` (bool, 1)	SetStaffLayout
28	`CreateRepeatStructure`	lp(`RepeatStructure`) — the schema-major-2 layout, unconditionally (born at v2; Section 9.5)	Repeat Structures
29	`DeleteRepeatStructure`	`repeat` (16)	Repeat Structures
30	`TransposeInterval`	seq⇑(`PitchId`) ‖ `diatonic_steps` (i32 LE, 4) ‖ `chromatic_steps` (i32 LE, 4)	TransposeInterval
31	`CreateInstrument`	lp(`Instrument`) — the schema-major-2 layout, unconditionally (born at v2; Section 9.5)	CreateInstrument
32	`SetCanvasLayoutDefaults`	lp(`CanvasLayoutDefaults`)	SetCanvasLayoutDefaults
33	`SetSpellingPrecedence`	lp(`SpellingPrecedence`)	SetSpellingPrecedence

Disc	Kind	Payload layout	Catalog section
34	`SetTuningContext`	lp(`TuningContextSettings`) — the **authored subset** of `ScoreTuningContext`: exactly its five wire-bearing fields, in the same order, byte-identical to the schema-major-3 `ScoreTuningContext` layout (Section 9.6); born at v3 unconditionally	SETTUNINGCONTEXT
35	`CreateStaffGroup`	lp(`StaffGroup`)	CREATESTAFFGROUP
36	`CreatePartDefinition`	lp(`PartDefinition`)	CREATEPARTDEFINITION
37	`CreateAnalysisLayer`	lp(`AnalysisLayer`)	CREATEANALYSISLAYER
38	`CreateView`	lp(`ViewDefinition`)	CREATEVIEW
39	`CreateMeasure`	`instance` (16) ‖ lp(`Measure`)	CREATEMEASURE
40	`CreateTuplet`	lp(`Tuplet`)	CREATETUPLET
41	`SetClef`	`instance` (16) ‖ lp(`RationalTime`) ‖ option tag (0/1) ‖ lp(`Clef`) if present	SETCLEF AND SETKEYSIGNATURE
42	`SetKeySignature`	`instance` (16) ‖ lp(`RationalTime`) ‖ option tag (0/1) ‖ lp(`KeySignature`) if present	SETCLEF AND SETKEYSIGNATURE
43	`SetVoiceHome`	`voice` (16) ‖ option tag (0/1) ‖ `home` (`StaffId`, 16) if present	SETVOICEHOME

Sub-vocabularies used above (one discriminant byte, then the listed payload):

`TupletCompensation`. 0 NotInTuplet (empty); 1 ReplaceWithRest ‖ lp(`Rest`); 2 RewriteTuplets ‖ seq$^{\uparrow}$(`TupletId`); 3 CascadeDeleteTuplets ‖ seq$^{\uparrow}$(`TupletId`).

`CrossCuttingValue`. 0 Tie, 1 Slur, 2 Beam, 3 Spanner, 4 Marker and 5 Lyric (both from schema major 5, Section 9.8); in every case the tag is followed by lp(the typed structure's `CanonicalValue` bytes).

`PositionRemapping`. 0 PreserveTime (empty); 1 Reassign ‖ `u32` LE entry count ‖ entries, each `EventId` (16, raw) ‖ lp(`MusicalPosition`), ascending by `EventId`.

`TransactionCategory`. 0 NoteEntry, 1 Structural, 2 Layout, 3 Import, 4 Registered ‖ `OperationKindRegistryId` (16 BE) — governed by `app:bytes` (core requirement `req:semops:transaction-category`).

`OperationKind` is encode-only at the storage boundary, like the envelope that carries it.

6.4 `OperationKindTag`: a Separate Discriminant Space

REQUIREMENT 6.3

`OperationKindTag` — the payload-free projection of an operation kind, stored by edit barriers (Chapter 8) — has its **own** discriminant space, assigned independently of Section 6.3, with both encode and validating decode. One byte; only `Registered` carries a payload (its `OperationKindRegistryId`, 16 big-endian bytes, total 17). Unknown discriminants and wrong lengths are decode errors. Append-only past 29.

Disc	Tag	Disc	Tag
0	`InsertEvent`	12	`DeleteStaffInstance`
1	`DeleteEvent`	13	`SetUserSystemBreak`
2	`ModifyEvent`	14	`SetUserPageBreak`
3	`RespellPitch`	15	`DeclareTransaction`
4	`Transpose`	16	`Registered`
5	`CreateCrossCutting`	17	`InsertIdentifiedPitch`
6	`DeleteCrossCutting`	18	`DeleteIdentifiedPitch`
7	`ModifyCrossCutting`	19	`ModifyIdentifiedPitch`
8	`ChangeRegionTimeModel`	20	`CreateVoice`
9	`InsertRegion`	21	`DeleteVoice`
10	`DeleteRegion`	22	`SetMetadata`
11	`InsertStaffInstance`	23	`SetMetricGrid`
24	`InsertStaff`	25	`SetTimeSignature`
26	`SetTempoSegment`	27	`SetStaffLayout`
28	`CreateRepeatStructure`	29	`DeleteRepeatStructure`
30	`TransposeInterval`	31	`CreateInstrument`
32	`SetCanvasLayoutDefaults`	33	`SetSpellingPrecedence`
34	`SetTuningContext`	35	`CreateStaffGroup`
36	`CreatePartDefinition`	37	`CreateAnalysisLayer`
38	`CreateView`	39	`CreateMeasure`
40	`CreateTuplet`	41	`SetClef`
42	`SetKeySignature`	43	`SetVoiceHome`

Naming mapping. The kind-to-tag projection renames three pairs: `OperationKind::CreateRegion` projects to `OperationKindTag::InsertRegion`, and `OperationKind::CreateStaffInstance` projects to `OperationKindTag::InsertStaffInstance`, and `OperationKind::CreateStaff` projects to `OperationKindTag::InsertStaff`. The mismatch is intentional (the tag space predates the M2c naming) and is pinned here so barrier authors target the right tag. The repeat-structure pair projects name-verbatim (`CreateRepeatStructure/DeleteRepeatStructure`), as the cross-cutting tags do.

6.5 Stamps and Causal Contexts

`HybridLogicalClock`: `physical_time` (`WallClockTime`, `i64` LE, 8) ‖ `logical_counter` (`u32` LE, 4). `OperationStamp`: `hlc` (12) ‖ `id` (`OperationId`, 16 BE) — **28 bytes** total, fixed width. The canonical reduction tuple orders stamps as (*physical_time*, *logical_counter*, *replica*, *counter*), ascending; the replica comparison equals big-endian byte order.

`CausalContext` (dotted version vector):

Field	Encoding
vector	`u32` LE entry count ‖ entries, each `replica` (8 BE) ‖ `counter` (`u64` LE, 8), ascending by replica
dots	`u32` LE count ‖ `OperationId`s (16 BE each), ascending

The vector is *zero-based-floor* (P11-C7): an entry (r, c) covers counters $0..=c$ of replica r; absence of a replica covers nothing.

6.6 Effects, Conflicts, Anomalies, and Materialized State

These types are canonical *materialized* facts: they appear in `MaterializedState`'s canonical bytes, so their discriminants and layouts are normative wire facts even though they are never stored inside envelopes. All decode validation is performed by the ops layer's validating decoder; unknown discriminants, bad lengths, non-canonical orders, and trailing bytes are decode errors.

Type	Layout (tag byte, then payload)
`OperationEffect`	0 Applied (empty); 1 AppliedWithRepair ‖ seq(`RepairRecord`); 2 Conflicted ‖ `ConflictId` (16 BE); 3 TombstonedTarget ‖ `TypedObjectId`; 4 NoOp ‖ `NoOpReason`.
`NoOpReason`	0 TargetTombstoned; 1 AlreadyApplied; 2 SupersededByLaterOperation ‖ `OperationId` (16 BE); 3 PreconditionFailedUnderReduction ‖ `PreconditionFailureReason`; 4 TransactionConflict.
`Precondition FailureReason`	0 TargetMissing; 1 TargetTombstoned; 2 WrongRegionTimeModel; 3 TupletCompensationInvalid; 4 EventDurationInvalid; 5 PositionOutsideRegion; 6 PitchSpaceMismatch; 7 VoiceMissing; 8 ExtensionPrecondition ‖ id (16 BE); 9 Registered ‖ id (16 BE); 10 ContainerNotEmpty; 11 TempoMapMalformed; 12 SystemDerivedContentImmutable (Pass 12, P12-K3); 13 RecreateContentMismatch (Pass 12, P12-K9); 14 AcousticRealizationPinned; 15 TranspositionOutOfRange (both Push 4a); 16 MeasureMeterMismatch, 17 MeasureOutOfOrder, 18 MeasureOrderUnverifiable (genesis tranche G3b); 19 SlotOccupied, 20 HomeIsOwnStaff (both X4b).
`RepairRecord`	(struct) `kind` (`RepairKind`) ‖ `target` (`TypedObjectId`).
`RepairKind`	0 Reanchored ‖ `from` (`TypedObjectId`) ‖ `to` (`TypedObjectId`) ‖ `ReanchorReason`; 1 SpannerTruncated ‖ seq(`TypedObjectId`); 2 Orphaned; 3 CascadeDeleted; 4 AttachmentTombstoned; 5 VoicePromoted ‖ `from` (`VoiceId`, 16 BE) ‖ `to` (`VoiceId`, 16 BE); 6 TupletCompensated ‖ `TupletCompensationKind`; 7 Registered ‖ id (16 BE).
`ReanchorReason`	0 SameVoiceNearer; 1 SameStaffInstanceNearer; 2 SameStaffNearer; 3 SameRegionNearer; 4 ExplicitFallback; 5 DeclaredByExtension ‖ id (16 BE); 6 SameCanvasNearer (Pass 12, P12-C4: semantically the rank-4 proximity variant; appended at 6 because 5 was already owned by DeclaredByExtension).
`TupletCompensationKind`	0 ReplaceWithRest; 1 RewriteTuplets; 2 CascadeDeleteTuplets. (No payload; distinct from the payload-carrying `TupletCompensation` of Section 6.3.)
`PendingReason`	tag ‖ blocker (`OperationId`, 16 BE) in every variant: 0 MissingCausalPredecessor; 1 DependsOnEquivocated; 2 DependsOnExcluded; 3 DependsOnPending; 4 HaltedBySystemCollision.
`ConflictKind`	0 StructuralFieldCollision ‖ `winner` (16 BE) ‖ `loser` (16 BE) ‖ `FieldPath` (NFC string); 1 TransactionConflict ‖ `transaction` (16 BE) ‖ seq$^{\uparrow}$(`OperationId`); 2 TombstonedTarget ‖ `TypedObjectId` ‖ `OperationId` (16 BE); 3 ReanchorFailure ‖ `original_referent` (`TypedObjectId`) ‖ `referencing_object` (`TypedObjectId`); 4 TimeModelMigrationFailure ‖ `region` (16 BE) ‖ seq$^{\uparrow}$(`TypedObjectId`); 5 ExtensionConflict ‖ `kind_id` (16 BE) ‖ lp(opaque details).
`ConflictResolutionState`	0 Unresolved; 1 Resolved ‖ `by` (16 BE) ‖ `ResolutionAction`; 2 Dismissed ‖ `by` (16 BE).
`ConflictRecord`	(struct) `id` (`ConflictId`, 16 BE) ‖ seq(`caused_by`: `OperationId`, stored sorted) ‖ `kind` ‖ seq(`affected_objects`: `TypedObjectId`, stored sorted) ‖ `resolution_state`.
`IntegrityAnomaly`	(struct) `id` (`IntegrityAnomalyId`, 16 BE) ‖ `kind`.

Type	Layout (tag byte, then payload)
`IntegrityAnomalyKind`	0 SystemIdentifierCollision ‖ `ObjectKind` ‖ `colliding_counter` (`u64` LE) ‖ the two input-set blobs, each lp(bytes), emitted lo ≤ hi by byte comparison; 1 OperationSlotEquivocated ‖ `OperationId` (16 BE); 2 ReplicaStreamQuarantined ‖ `replica` (8 BE) ‖ `first_bad_counter` (`u64` LE); 3 Registered ‖ id (16 BE).
`AnomalousReplicaSegment`	(struct) `replica` (8 BE) ‖ `first_bad_counter` (`u64` LE) ‖ `ReplicaAnomalyReason` (0 HlcMonotonicityViolation ‖ two `OperationId`s; 1 Registered ‖ id 16 BE) ‖ seq(`excluded`: `OperationId`, ascending counter).
`FieldPath`	an NFC string (`u32` LE length ‖ UTF-8).
`ObjectState`	0 Live (empty); 1 Tombstoned ‖ `deleted_by` (`OperationId`, 16 BE) ‖ `minted_by` (`OperationId`, 16 BE).
`ObjectKind` (ops)	0 Voice; 1 Pitch; 2 Registered ‖ `OperationKindRegistryId` (16 BE) — governed by `app:bytes` (core requirement `req:graph:object-kind-vocab`). Distinct from the barrier layer's 2-byte `ObjectKind` (Chapter 8).

6.6.1 `MaterializedState`

The canonical bytes of a reduction's materialized state are eight sections, concatenated; every section is `u32`-LE-counted (the conflict registry carries its own count) and in the stated normative order, so the bytes are identical across any permutation of the input operation set:

1. **effects** — count, then per entry `OperationId` (16 BE, raw) ‖ lp(`OperationEffect`); in canonical reduction order;
2. **conflict registry** — a seq of lp(`ConflictRecord`), ascending `ConflictId`;
3. **anomalies** — count, then each lp(`IntegrityAnomaly`), ascending `IntegrityAnomalyId`;
4. **objects** — count, then per entry `TypedObjectId` (raw) ‖ `ObjectState`, ascending `TypedObjectId`;
5. **spellings** — count, then per entry `PitchId` (16 BE, raw) ‖ lp(`PitchSpelling`), ascending `PitchId`;
6. **breaks** — count, then per entry `RegionId` (16 BE, raw) ‖ `MusicalPosition` (raw `RationalTime` form, self-describing, *not* leaf-framed) ‖ `present` (bool, 1); ascending (*region*, *position*);
7. **page breaks** — same shape and order as breaks;
8. **pending** — count, then per entry `OperationId` (16 BE, raw) ‖ `PendingReason`, ascending `OperationId`.

Decode is fully validating (orders, tags, lengths, trailing bytes) and rejects non-canonical input.

6.7 v0 Payloads Have No Wire Form

The prototype's v0 identifier-projection payload types are an **in-memory migration guard only**: they carry no canonical encoding, and no v0 wire form was ever shipped. The v0→v1 migration is *semantic* — typed value reconstruction, specified in the Operation Catalog's v0 → v1 PAYLOAD MIGRATION chapter — not a byte-level translation, and nothing in this document defines bytes for a v0 payload.

Chapter 7

Bundle Physical Layout

The physical file: the fixed prelude, chunks, blocks, the manifest, and the operation index. This chapter **ratifies** the bundle crate's provisional encodings (its `DECISIONS` entries P11-D2, P11-D4, and P11-D5 pending-companion status is discharged) and the operation-index byte form (P12-D1). Structural *semantics* — superblock selection, commit protocol, recovery, retention — are core specification Chapter 8; this chapter pins bytes only. Regime (a) applies except where a table says otherwise.

7.1 The Fixed Header (64 bytes)

Offset 0 of every bundle. Written at creation, never rewritten within a format major version. All integers little-endian.

Range	Field	Value / rule
0..8	magic	`MUSCBND\0` (8 bytes, ASCII, trailing NUL)
8..10	`format_major` (`u16`)	1 written in this version. The container *epoch*: a major-1 container is one whose every base-bearing commit was validated against a supplied reduction authority. 0 is *legacy* and **MUST** be decoded deliberately as such, not rejected; any other value is rejected. A legacy container **MAY** be opened only while it carries no canonical base, and **MUST NOT** acquire one — the epoch is fixed at creation and cannot be inherited by a later commit (Core Specification, The Container Epoch)
10..12	`format_minor` (`u16`)	0 written in this version; minor numbering restarts at a new major
12..16	`header_length` (`u32`)	64; readers reject other values
16..24	`superblock_a_offset` (`u64`)	64; readers reject other values
24..32	`superblock_b_offset` (`u64`)	320; readers reject other values
32..48	`file_uuid`	16 opaque bytes
48..60	reserved	written zero; ignored on read
60..64	`header_crc`	CRC-32C of bytes 0..60, `u32` LE

Readers **MUST** verify the magic and the CRC before consulting any other part of the file.

7.2 The Superblock Slots (256 bytes each)

Two slots, at offsets 64 (A) and 320 (B); the only mutable on-disk objects. Selection *semantics* (highest valid committed generation, the equal-generation rule, torn-write fallback) are core Chapter 8, Superblock Selection; the bytes:

Range	Field	Encoding
0..8	magic	`MUSCSUPR` (8 ASCII bytes)
8..16	`generation`	`u64` LE
16..24	`manifest_offset`	`u64` LE
24..32	`manifest_length`	`u64` LE (also the uncompressed length; the manifest is mandatorily uncompressed)
32..64	`manifest_hash`	32 raw digest bytes
64..68	`manifest_schema_version`	`SchemaVersion` (`u16` LE ‖ `u16` LE)
68..72	`reduction_algorithm_version`	`u32` LE
72..92	`profile_id`	20 bytes: `u32` LE discriminant (0 Full, 1 ReadOnly, 2 Lite, 3 Custom) ‖ `ProfileRegistryId` (16 raw bytes; writers **MUST** emit zero unless Custom, readers ignore it unless Custom). Core requirement `req:format:profileid-discriminants`.
92..100	`commit_state`	two `u32` LE words: Committed $= (0, 0)$; any tag $\neq 0$ decodes as Reserved(value) and the slot is invalid for ordinary selection. Writers in this version **MUST** produce only Committed.
100..108	`commit_timestamp`	`i64` LE nanoseconds; advisory — selection never consults it
108..252	reserved	written zero; ignored on read
252..256	`superblock_crc`	CRC-32C of bytes 0..252, `u32` LE

7.3 Chunks

The bundle body begins at `BODY_START` $= 320 + 256 = 576$. A chunk's on-disk form is its raw (possibly compressed) payload bytes at a byte offset — **no per-chunk length prefix, magic, or trailer on disk**; the offset, lengths, compression, and hash live in the referencing `ChunkRef`. A chunk reference whose offset is below `BODY_START` is malformed.

Content hash preimage (core Chapter 8, Domain-Separated Preimages; governs identity):

$$\text{BLAKE3}(\textit{domain} \,\|\, \textit{kind}\ (1) \,\|\, \textit{schema}\ (4) \,\|\, \textit{uncompressed_length}\ (\texttt{u64}\ \text{LE}) \,\|\, \textit{payload})$$

where *domain* is `MUSCMANI` for `Manifest` chunks and `MUSCCHNK` otherwise. Compression is *not* in the preimage. Blob chunks are addressed by the bare `BlobId` form instead (Chapter 4).

`ChunkKind` is a single byte, a **closed** vocabulary (no `Registered` variant; core requirement `req:format:chunkkind-discriminants`): 0 OperationEnvelopeBlock, 1 OperationIndex, 2 Snapshot, 3 Blob, 4 ExtensionData, 5 TextProjection, 6 LayoutCache, 7 IntegrityIndex, 8 Manifest.

`CompressionAlgorithm` is a fixed two bytes, discriminant ‖ parameter: None $=(0,0)$; Zstd{level} $=(1, level)$; Reserved$(v)=(2,v)$. `None` is *not* a bare tag — its zero parameter byte is always present.

REQUIREMENT 7.1

A decoder **MUST** reject a `CompressionAlgorithm` whose discriminant is 0 (`None`) and whose parameter byte is non-zero. It **MUST NOT** ignore the byte, and **MUST NOT** normalize it to zero.

RATIONALE

Ignoring it made the codec *lenient*, and therefore non-injective: $(0,255)$ and $(0,0)$ decoded to one value, which re-encodes to the latter. Every structure embedding a `ChunkRef` inherited that. A whole-value re-encode-and-compare guard hides it — the manifest has one — but the operation index does not, and so accepted two distinct payloads for one index while its own contract promised to reject non-canonical bytes rather than normalize them. Content-addressing rests on distinct bytes meaning distinct values.
No conforming writer emits a non-zero parameter for `None`, so this rejects only corrupt or adversarial input; no existing file changes meaning. Found by the wire-decode fuzzer (Push 5, P3); supersedes the core specification's earlier "present but zero, and ignored on read".

Read rules. Writers in this format version emit `None` only; *reading* zstd at any level is a conformance **MUST**. A zstd payload is decompressed into a buffer sized *exactly* by the declared `uncompressed_length` (checked against the reader's resource policy before allocation): a stream that would exceed the declaration, ends short of it, or carries trailing garbage is corruption. After decompression the content hash is verified over the uncompressed bytes, and $id = hash$ is checked. A chunk whose declared schema major differs from the reader's supported major is rejected (Chapter 9). `Reserved` compression is rejected. The **manifest chunk is mandatorily uncompressed**: a compressed manifest reference is rejected before any bytes are read. The reference reader bounds any single chunk at 256 MiB (`MAX_CHUNK_BYTES`); the bound is reader policy, not a wire fact.

7.3.1 `ChunkRef` (95 bytes)

Range	Field	Encoding
0..32	`id`	32 raw digest bytes
32..33	`kind`	`ChunkKind` byte
33..37	`schema_version`	`u16` LE ‖ `u16` LE
37..45	`offset`	`u64` LE
45..53	`compressed_length`	`u64` LE
53..61	`uncompressed_length`	`u64` LE
61..63	`compression`	2 bytes
63..95	`hash`	32 raw digest bytes (restates `id`)

The canonical sort key for chunk-reference collections is (*kind discriminant*, *hash*, *offset*), ascending — the Appendix-D chunk-reference order. Every set-valued `ChunkRef` field in this chapter is sorted by it and deduplicated before encoding.

7.4 Operation-Envelope Block Payloads

The uncompressed payload of an `OperationEnvelopeBlock` chunk:

`u32` LE envelope count ‖ per envelope {`u32` LE byte length ‖ envelope bytes}

Envelope bytes are opaque to the bundle (their internal form is Chapter 6). Each envelope's offset — the byte offset of its *first content byte* within the uncompressed block payload, with its length prefix at *offset* − 4 — is deterministically recoverable from this framing alone; it is the coordinate the operation index records. Writers **SHOULD** begin a new block rather than let an uncompressed payload exceed 1 MiB (an individual envelope larger than that occupies its own block); readers **MUST** accept blocks up to the active profile's bound (default 64 MiB). Block boundaries are storage artifacts: the envelope set is the union across blocks.

7.5 The Manifest

On-disk manifest chunk payload:

`manifest_id` (16 bytes, `u128` *little-endian*) ‖ *body*

(The little-endian on-disk field is a deliberate, golden-locked quirk: the *derivation* truncates big-endian like every content-derived id, but the bundle codec serializes the resulting `u128` under its little-endian integer rule.)

REQUIREMENT 7.2

Restating core requirement `req:format:manifest-id`, which governs:

$$\texttt{ManifestId} = \text{trunc128}\big(\text{BLAKE3}(\texttt{MUSCMNIF} \,\|\, \textit{document_id} \,\|\, \textit{generation} \,\|\, \textit{body})\big)$$

with *document_id* as its 16 raw bytes, *generation* as `u64` little-endian, and *body* the complete manifest body below (which itself opens with `document_id` and `generation` — the double commitment is intentional and locked). Writers re-derive the id from the body at encode time; decoders verify it.

The body is a positional struct of thirteen fields:

#	Field	Encoding and canonical order
1	`document_id`	16 raw bytes
2	`lineage_id`	option tag ‖ 16 raw bytes if present
3	`generation`	`u64` LE
4	`operation_roots`	`u32` LE count ‖ `ChunkRef`s, sorted by the canonical chunk-reference key, deduplicated
5	`operation_block_summaries`	`u32` LE count ‖ entries {`chunk_id` (32) ‖ `OperationBlockSummary`}, ascending `chunk_id`

#	Field	Encoding and canonical order
6	`operation_index_root`	option tag ‖ `ChunkRef`
7	`canonical_base`	option tag ‖ `SnapshotRef`
8	`acceleration_snapshots`	`u32` LE count ‖ `SnapshotRef`s, sorted ascending by their full encoded bytes, deduplicated
9	`blob_roots`	`u32` LE count ‖ `BlobRef`s, sorted ascending by encoded bytes, deduplicated
10	`profile_declarations`	`u32` LE count ‖ `ProfileDeclaration`s, sorted by the key (`profile_id`, `version`) with `SemVer` compared *numerically*, deduplicated
11	`extension_declarations`	`u32` LE count ‖ `ExtensionDeclaration`s, sorted by (`extension_id`, `version`), `SemVer` numeric, deduplicated
12	`text_projection_root`	option tag ‖ `ChunkRef`
13	`integrity_root`	option tag ‖ `ChunkRef`

Sub-records, positionally:

`OperationBlockSummary` = `dvv_summary` (`FrontierBytes`: `u32`-LE-prefixed opaque bytes) ‖ `min_stamp` (`u32`-LE-prefixed opaque stamp bytes) ‖ `max_stamp` (same). The contents are ops-computed (Chapter 6); the bundle carries them verbatim.

`SnapshotRef` = `snapshot_id` (16 raw) ‖ `covers_causal_frontier` (`FrontierBytes`) ‖ `reduction_algorithm_version` (`u32` LE) ‖ `profile_id` (20 bytes, as in the superblock) ‖ `root` (`ChunkRef`, 95) ‖ `hash` (32 raw).

`BlobRef` = `blob_id` (32 raw) ‖ `media_type` (`u32`-LE-prefixed ASCII; decode validates a well-formed RFC 6838 `type/subtype` restricted name) ‖ `offset` (`u64` LE) ‖ `compressed_length` (`u64` LE) ‖ `uncompressed_length` (`u64` LE) ‖ `compression` (2) ‖ `hash` (32 raw) ‖ option tag ‖ `declared_max_uncompressed_length` (`u64` LE) if present.

`ProfileDeclaration` = `profile_id` (20) ‖ `version` (`SemVer`, 12) ‖ `constraints` (`ProfileConstraints` = `max_uncompressed_block_size` (`u64` LE) ‖ `RetentionPolicy`). `RetentionPolicy` = `retain_previous_manifests` (`u32` LE) ‖ option tag ‖ `retain_duration` (`i64` LE) if present ‖ `retain_named_checkpoints` (bool, 1).

`ExtensionDeclaration` = `extension_id` (16 raw) ‖ `version` (`SemVer`, 12) ‖ `required` (bool, 1) ‖ `u32` LE count ‖ `preserved_chunk_roots` (`ChunkRef`s, canonical order, deduplicated) ‖ `affected_object_kinds` (`u32`-LE-prefixed opaque blob) ‖ `edit_barriers` (`u32`-LE-prefixed opaque blob). The two blobs' internal form is Chapter 8; the bundle preserves them verbatim.

REQUIREMENT 7.3

Reject non-canonical manifests. A decoder **MUST** accept a manifest payload only if re-encoding the decoded manifest reproduces the input byte-for-byte. This single check subsumes: the stored `manifest_id` matching the body-derived id, every set-valued vector being in its canonical order with no duplicates, and every optional field using presence byte 0/1. Unsorted, duplicated, or wrongly-identified manifest bytes are malformed — never re-sorted or repaired.

7.6 The Operation Index

REQUIREMENT 7.4

Operation-index payload (ratifies P12-D1). The uncompressed payload of an `OperationIndex` chunk is:

`u32` LE `block_count` ‖ that many `ChunkRef`s (95 bytes each, strictly ascending canonical order) ‖ `u32` LE `entry_count` ‖ that many entries, each {`id` (16 raw operation-id bytes) ‖ `block` (`u32` LE ordinal into the block vector) ‖ `offset` (`u32` LE)}, strictly ascending by `id` bytes.

The empty index is exactly 8 zero bytes. `offset` is the byte offset of the envelope's *first content byte* within the uncompressed payload of the named block (the envelope's `u32` length prefix sits at *offset* − 4) — well-defined by the id-leads property (Requirement 6.1) and the block framing (Section 7.4).

One slot per id. Strict ascent makes duplicate ids unrepresentable: an index payload containing two entries with equal id bytes is malformed. Equivocation candidates (multiple envelopes claiming one `OperationId`) live *inside* the operation set's slot model (core Chapter 6), never as duplicate index rows.

Staleness. An index *covers* a manifest iff its block vector equals the manifest's deduplicated, canonically sorted `operation_roots` under **full `ChunkRef` equality** — every field, not just the content hash, since the index hands out its stored references for reading. A reader that finds the index stale **MUST** reject it and rebuild from the blocks.

A rejected or stale index is **never bundle corruption**: the index is a non-canonical accelerator, and rebuild-from-blocks is always sound.

Decoders **MUST** reject: unsorted or duplicated blocks or entry ids, a block reference whose kind is not `OperationEnvelopeBlock`, an entry whose block ordinal is out of range, and trailing bytes.

The core specification's commit-time **SHOULD** — refresh the index when the operation log has "grown significantly" since it was built — has an **implementation-defined** threshold in v0: this document deliberately does not pin a number, and conforming writers may use any policy (including always or never refreshing at commit), because the index is non-canonical.

OPEN QUESTION

Whether to pin a normative refresh threshold (e.g. a block-count or byte-size ratio) or leave it permanently implementation-defined. Leaving it open costs only lookup performance on stale-index bundles; pinning it too early would constrain writers for no interoperability gain. Revisit when a second writer implementation exists.

Chapter 8

Extension Declaration Blobs and Edit Barriers

The two opaque blobs a manifest `ExtensionDeclaration` carries — `affected_object_kinds` and `edit_barriers` — have their internal byte form pinned here. This chapter **ratifies** P12-E1 (the blob and barrier-tree byte form), P12-E2 (the recursion bound), and P12-E3 (the barrier `ObjectKind` representation). Barrier *semantics* — scope matching, prohibited-edit evaluation, conservative treatment of unknown extensions — are core Chapter 8 (Forward Compatibility and Edit Barriers); this chapter pins bytes only.

These layouts are **regime (b)**: all counts and length prefixes are `u64` little-endian — a deliberate, golden-locked divergence from the `u32` regime (Section 2.1). They **keep this form in schema major 1**: unlike the resolved-layout surface, these blobs are carried *opaquely inside the canonical manifest* (Section 7.5), which stays major 0, so a major-0 reader must continue to parse them as `u64`. Their unification waits for a future major that revises the manifest (Section 9.4).

Open Question

Unsafe-edit tombstone encoding (from P12-E5). Core Chapter 8 now ratifies the unsafe-edit tombstone *semantics* (immediate deactivation of the crossed extension's remaining barriers; a durable record **MUST** land at the next commit; `required = true` interaction). The manifest-side *byte encoding* of that record is unresolved — and constrained: the manifest schema is frozen at major 0, so the record cannot be a new manifest field. Candidate channels: a marker inside the extension-declaration blob layer (this chapter's regime-(b) surface), or a dedicated chunk kind referenced from `preserved_chunk_roots`. To be designed in the next bundle-format tranche; the editor-side producer (`extensions_requiring_tombstone()`) already exists.

8.1 Framing

element elem(x) = `u64` LE byte length ‖ x's canonical bytes.

set set(xs) = elements' canonical byte strings, sorted ascending byte-lexicographic and *deduplicated*; then `u64` LE element count ‖ each element as elem. Order and repetition in the source collection cannot affect the bytes; decoders reject unsorted or duplicated elements.

list list(xs) = `u64` LE count ‖ each element as elem, order-significant (used only for condition subtrees).

Requirement 8.1

The two blobs (ratifies P12-E1). The `affected_object_kinds` blob is set of barrier `ObjectKind` elements; the `edit_barriers` blob is set of `EditBarrier` elements. Both decode under reject-never-normalize with trailing bytes forbidden.

Requirement 8.2

Barrier `ObjectKind` (ratifies P12-E3). A barrier `ObjectKind` is the `TypedObjectId` discriminant of Section 3.2, encoded as **2 little-endian bytes**. Decode is **open-value**: every `u16` decodes successfully — an unknown kind (a future core kind or an extension-registered kind) is a *value*, not a decode branch, and simply never matches any object the reader knows. This is the format's forward-compatibility stance for barrier kinds: unknown kinds are preserved and re-emitted verbatim, never dropped or rejected.

8.2 `EditBarrier`

An `EditBarrier`'s canonical bytes, positionally:

`scope` ‖ set(`affected_object_kinds`: `ObjectKind`) ‖ set(`prohibited_operation_kinds`: `OperationKindTag`, Section 6.4) ‖ `condition`

`BarrierScope`: one discriminant byte, then the payload:

Disc	Variant	Payload
0	`WholeScore`	(none)
1	`Region`	`RegionId`, 16 big-endian bytes
2	`StaffInstance`	`StaffInstanceId`, 16 BE
3	`AnalysisLayer`	`AnalysisLayerId`, 16 BE
4	`ObjectSet`	set of `TypedObjectId` canonical bytes (18/34 each)
5	`PitchSpace`	`u64` LE byte length ‖ the pitch-space id's NFC UTF-8 bytes; decoders reject non-NFC input
6	`TuningContext`	(none)
7	`Registered`	`BarrierScopeRegistryId`, `u128` *little-endian* (16 bytes)

`BarrierCondition`: one discriminant byte, then the payload; the tree is recursive:

Disc	Variant	Payload
0	`Always`	(none)
1	`ObjectExists`	`TypedObjectId`
2	`ObjectHasExtensionData`	`object` (`TypedObjectId`) ‖ `extension` (`ExtensionRef`, `u128` LE, 16)
3	`All`	list of conditions
4	`Any`	list of conditions

Disc	Variant	Payload
5	`Not`	one elem-framed condition
6	`Registered`	`BarrierConditionRegistryId`, `u128` LE (16)

Note the endianness: the barrier layer's registry identifiers (`BarrierScopeRegistryId`, `BarrierConditionRegistryId`, `ExtensionRef`) encode `u128` *little-endian*, unlike the big-endian identifier family of Chapter 3. This is part of the golden-locked regime-(b) surface.

Requirement 8.3

Recursion bound (ratifies P12-E2). `MAX_CONDITION_DEPTH` = 64 is the normative bound on `BarrierCondition` nesting. Decoders **MUST** reject a condition tree nested deeper than 64 levels of `All`/`Any`/`Not`; writers **MUST NOT** emit one. The bound exists so adversarial bytes cannot drive unbounded recursion; real barriers are depth 1–2.

Chapter 9

Schema Evolution

The core specification delegates schema evolution to this document (core Chapter 8, Schema Versioning: *"Schema evolution is governed by the Binary Format companion specification, which defines the wire encoding for each schema version"*). This chapter defines the wire rules for schema evolution. Chapters 5–8 specify the schema-major-0 layouts; Section 9.4 specifies the schema-major-1 delta and the migration between them, Section 9.5 the schema-major-2 delta and its migration from major 1, and Section 9.6 the schema-major-3 delta and its migration from major 2.

9.1 The Chunk-Level Gate

Every chunk declares a `SchemaVersion` (major, minor) in its `ChunkRef` and the superblock declares the manifest's. The gate:

- **Major** = incompatible. A reader **MUST** reject a chunk whose schema major it does not support. A reader supports a contiguous accept-set $[\text{MIN}, \text{MAX}]$ of majors and rejects any chunk whose major falls *outside* $[\text{MIN}, \text{MAX}]$ — too new (above MAX) or, once MIN rises past a retired major, too old (Section 9.4). Six majors are defined: 0, 1 (Section 9.4), 2 (Section 9.5), 3 (Section 9.6), 4 (Section 9.7), and 5 (Section 9.8). The accept-set is **per chunk role**, not one shared bound (Section 9.4's "raised per chunk role" rule, sharpened by major 3, the first bump under which the roles' maxima genuinely diverge): the `Snapshot` role's max is 5; the `OperationEnvelopeBlock` role's max is 5 as of X4b (an operation whose value holds a value only major 5 encodes is born at v5; the role was 4 from X3c, when `CreateTuplet`, 40, was born at v4 unconditionally, 3 from genesis tranche G2b, `spec/CONTRACT_GENESIS_G2B_TUNING.md`, when `SetTuningContext`, 34, was born at v3 unconditionally, and 2 from Push 4b tranche 3b-i, when schema major 3 embedded no operation payload, until that rung); every other role's max is 0. The reference implementation's overall envelope is $\{0, 1, 2, 3, 4, 5\}$ ($\text{MIN} = 0$, $\text{MAX} = 5$), realized as these differing per-role maxima rather than a single shared bound.
- **Minor** = additive. v0 readers verify the major only; the minor is a *record*, not a gate — but it is a mandatory record: a writer **MUST** raise the chunk schema minor when it emits any discriminant appended after the minor it otherwise declares, so that a decode failure on an unknown appended discriminant is attributable to a version skew rather than corruption.

9.2 What "Additive" Means Here

Under the frozen-layout rule (Requirement 5.3), positional struct layouts admit *no* additive change: there is no optional-field or trailing-field mechanism at the value level. In particular, and stated honestly: **the manifest body cannot grow a fourteenth field within major 0**. The manifest is a positional struct whose decoder rejects trailing bytes, so manifest-body extension is major-gated in v0 exactly like every other struct.

Adding a field is a MAJOR change *regardless of its type*: an `Option` field is not "optional" in the wire sense — it still occupies a new positional slot (a presence byte, then the payload when present), which shifts every subsequent field and rejects under the trailing-bytes rule. There is no "downgrade to minor" for an `Option` addition. The first exercise of this is schema major 1 (Section 9.4), which adds `Instrument.range` (an `Option`) among other fields.

The *only* minor-additive mechanism in schema major 0 is **appending discriminants to open vocabularies**:

- `OperationKind`: append at ≥ 44 (Requirement 6.2; the Phase-3 tranche took 24–27 under this mechanism, the repeat pair 28/29 in the schema-major-2 revision, Push 4a took 30 (`TransposeInterval`), genesis tranche G1 took 31 (`CreateInstrument`), genesis tranche G2a took 32/33 (`SetCanvasLayoutDefaults`, `SetSpellingPrecedence`), genesis tranche G2b took 34 (`SetTuningContext`), genesis tranche G3a took 35–38 (`CreateStaffGroup`, `CreatePartDefinition`, `CreateAnalysisLayer`, `CreateView`), genesis tranche G3b took 39 (`CreateMeasure`), X3.1 took 40 (`CreateTuplet`), X3.6 took 41/42 (`SetClef`, `SetKeySignature`), and X4b took 43 (`SetVoiceHome`) — the discriminant space is one append-only table across majors, but a kind's *block stamp* follows minimal stamping over its payload: 28's payload embeds a v2 layout, so its blocks stamp major 2, Section 9.5; 34's payload embeds a v3 layout unconditionally, so its blocks stamp major 3, Section 9.6; 40's payload embeds the v4 `Tuplet` unconditionally since X3c, so its blocks stamp major 4, Section 9.7; 35–39 and 41–43 are each major 0, no versioned walk existing for any of their carried types; 0, 3, 8 and 14 stamp major 5 by value, Section 9.8);
- `OperationKindTag`: append at ≥ 44 (Requirement 6.3; the same twenty variants as `OperationKind`, at the same twenty discriminants, appended by the same eleven tranches);
- `OperationPayload`: append at ≥ 4 (Section 6.2; `ResolveEquivocation` = 3, Push 3, is the one and only append this vocabulary has had);
- the value-layer unions enumerated as closed in Requirement 5.3 may gain appended variants only through a ratified revision of this document, also minor-additive.

The schema-minor derivation (G-minor). Every appended discriminant above is annotated with a *global additive epoch*: one schema-minor value shared across every vocabulary, assigned in the order the revision that introduced it landed (`spec/PLAN_GMINOR_SCHEMA_MINOR.md` §4). An envelope's required minor is the maximum epoch over every discriminant it actually emits — its outer `OperationPayload` variant, its primitive `OperationKind` when `Primitive`, and any nested additive variant its encoder reaches; a block's is the maximum over its envelopes. The emitted version is $\{\text{major}, \max(\text{baseline_minor}(\text{major}), \text{epoch_max})\}$: existing baselines (1 for major 0, 0 for majors 1–3) are never lowered, and a block emitting only baseline vocabulary keeps its major's

baseline minor rather than being normalized to zero. This is content-minimal per-role stamping, not a single document-wide counter: the canonical base, the manifest, and each operation block are each stamped from only the discriminants their own bytes emit (Section 9.2's per-chunk framing extends unchanged). The manifest reaches `OperationKindTag` independently of any operation envelope, through `ExtensionDeclaration.edit_barriers` → `EditBarrier.prohibited_operation_kinds`: a manifest naming a barrier on a post-baseline tag takes that tag's epoch, and the manifest major stays 0 regardless.

`ChunkKind` is **closed** — it has no `Registered` variant and no append story inside major 0, because its discriminant enters every chunk's hash preimage; a new chunk kind is a format-major event.

Separately, many vocabularies carry a `Registered` *escape variant* (`RepairKind`, `ReanchorReason`, `PreconditionFailureReason`, `IntegrityAnomalyKind`, `ReplicaAnomalyReason`, `TransactionCategory`, `ResolutionAction`, `ConflictKind` via `ExtensionConflict`, `BarrierScope`/`BarrierCondition`, `TieClass`, `StaffGroupKind`, `PitchSpacePosition`, `SpellingNominal`, `TypedObjectId`, and the barrier `ObjectKind`'s open value space). Extension through an escape variant is **not** a schema change at all: the wire form is already defined.

9.3 Migration Discipline

A schema-major bump **MUST** ship with a migration documented in this document's revision history (Chapter 12): what changed, byte-for-byte, and the deterministic translation from the old form. The precedent is the v0→v1 operation-payload migration, which lives in the Operation Catalog (v0 → v1 Payload Migration) because it was semantic rather than byte-level (Section 6.7); byte-level migrations belong here.

The `u64`/`u32` length-prefix divergence between regimes (a) and (b) (Section 2.1), left open in earlier revisions for a future major, is **partly resolved in schema major 1** (Section 9.4): the first major bump, scheduled for data-model reasons, carries the unification for the independent non-canonical resolved-layout surface. The barrier/extension blobs remain regime (b) because they ride the canonical manifest, which this bump keeps at major 0; their unification stays open for a manifest-revising major.

9.4 Schema Major 1

Schema major 1 is the first data-model expansion major, defined here in full: the changed value layouts, the length-prefix unification, and the byte-for-byte migration from major 0. It bundles every change that was deferred to "the next major" so that one coordinated migration discharges them together.

9.4.1 Where the changed fields reach

The three added fields land in different chunk classes, and the major is assigned **per payload type**, not per chunk kind:

- `Canvas.layout_defaults` and `Instrument.range` now also reach the **canonical operation layer**, each by a different genesis-tranche operation. `Instrument.range`: gene-

sis tranche G1's `CreateInstrument` (§6.3 discriminant 31) embeds the full `Instrument` value, whose schema-major-2 appends are mandatory — so this payload always carries the complete (major-2) `Instrument` layout, `range` included, and a block carrying it is stamped **major 2** unconditionally, never merely major 1. `Canvas.layout_defaults`: genesis tranche G2a's `SetCanvasLayoutDefaults` (discriminant 32) embeds the full `CanvasLayoutDefaults` value directly, rather than through the containing `Canvas` walk that carries the major-1 versioning — `CanvasLayoutDefaults` has exactly one layout as a standalone value, so a block carrying this operation is stamped **major 0**. What remains confined to the **non-canonical** acceleration-cache full-`Score` snapshot (a `ChunkKind::Snapshot`) is the canvas *object itself*: there is still no `CreateCanvas`, no `TypedObjectId` kind for it, and no operation mints, addresses, or deletes it — the canvas is the inline genesis singleton, and that half of the earlier claim survives.

- `Region.permits_spanning_slurs` reaches the snapshot *and* the **canonical operation layer**: `CreateRegion` embeds the full `Region` value (Section 6.2), so its v1 payload carries the v1 `Region` layout. An operation-envelope block containing a `CreateRegion` therefore encodes v1 bytes and is stamped **major 1**; a block with no such operation stays **major 0**. Operation-envelope blocks are canonical, so this is a canonical operation-layer change, not merely a cache change.
- `ChunkKind::Snapshot` is itself payload-polymorphic: the same kind carries *either* an acceleration-cache full `Score` (as above, stamped major 1) *or* the canonical-base `MaterializedState`, disambiguated by manifest role. The canonical base embeds no `Canvas`, `Instrument`, or `Region` value, so it is unchanged and stays **major 0**. Its content-hash preimage (Section 7.3) includes the schema version, so re-stamping the unchanged base at major 1 would change its chunk id and churn every `SnapshotRef` for zero layout benefit. A writer **MUST NOT** do so; a conformance test **SHOULD** assert the canonical base is byte-identical across the bump.

9.4.2 Cross-major reader behaviour

Two rules follow from the chunk classes above and the core specification's SCHEMA VERSIONING canonical/non-canonical distinction:

- **Non-canonical chunks — discard and regenerate.** The layout caches and the acceleration snapshot are non-canonical. A reader meeting a foreign-major one **MAY** discard and regenerate it rather than decode it: a major-1 reader regenerates a major-0 layout cache instead of carrying a frozen old-width decoder, and it migrates a major-0 acceleration `Score` snapshot on read (cheaper than replaying the operation log).
- **Canonical chunks — parse or open read-only.** The manifest and the canonical base stay major 0 and are always parseable. A major-1 op block is canonical: a major-0-only reader **MUST NOT** discard it (that would fork canonical state) — it opens the bundle in read-only preservation mode, having read the major-0 base and manifest but being unable to replay the v1 `CreateRegion` operations. A major-1 reader migrates such a block on read (below).

So a major-0-only reader opens a major-1 bundle *fully* only when the bundle contains no v1 `CreateRegion` operation; otherwise it opens read-only. It always reads the canonical base and manifest, and always discards the higher-major non-canonical caches.

9.4.3 The accept-set gate

A reader supports a contiguous set of majors [MIN, MAX] (at the major-1 revision the reference implementation's set was {0, 1}; the *current* set is stated once, in Section 9.1, and is not repeated here). The chunk-level gate rejects a chunk whose major falls outside [MIN, MAX] — too new above MAX, or too old below MIN once a reader drops support for a retired major; it no longer rejects on inequality with a single supported major.

9.4.4 Changed and new value layouts

These extend Chapter 5's regime (a) rules. As there, **the wire form is the reference implementation's struct layout** — which is a reduced subset of the core specification's fuller data model (the code's `Instrument` carries a subset of the model's fields, etc.). The wire form ratifies the code, not the model; a struct's v1 layout **appends** its new field(s) after its existing major-0 fields:

- `Canvas` = `regions` ‖ `layout_defaults`, where `CanvasLayoutDefaults` = `page_size` (`CanvasSize` = `width` ‖ `height`, two `CanonicalF64` leaves) ‖ `margins` (`CanvasMargins` = four `CanonicalF64` leaves: top, right, bottom, left).
- `Instrument` = `id` ‖ `name` ‖ `range` (`Option<PitchRange>`: a presence byte, then for `Some` a `PitchRange` = `lowest` ‖ `highest`, each a `Pitch` under this chapter's rules).
- `Region` appends `permits_spanning_slurs`, one bare bool byte (0/1), after its major-0 fields.
- **Operation layer:** the `CreateRegion` payload embeds a length-prefixed `Region` (Section 6.2), so its v1 form carries the v1 `Region` layout (the appended bool inside the embedded value). This is the one *canonical* byte change in major 1; the operation envelope's own field order is unchanged, only its embedded `Region` grows.

Every embedded leaf keeps this chapter's framing (a `CanonicalF64` is a `u32` LE length ‖ 8 bytes = 12; a bool is one bare byte).

9.4.5 Length-prefix unification (partial)

Regime (b) (Section 2.1) is *narrowed*, not fully retired, in major 1. The `ResolvedLayoutIR` output — an independent, non-canonical `LayoutCache` chunk — moves from `u64` to `u32` LE counts and length prefixes, matching regime (a), and its golden anchors re-lock in the same step. Because it is non-canonical, a major-1 reader regenerates a major-0 resolved-layout cache rather than decoding its `u64` form — no frozen `u64` decoder is required.

The extension-declaration and edit-barrier blobs (Chapter 8) **stay regime (b)**: they are carried opaquely inside the canonical manifest (Section 7.5), which stays major 0, so a major-0 reader must keep parsing them as `u64`. Unifying them would require bumping the manifest itself, which this data-model bump deliberately avoids; that unification waits for a manifest-revising major.

9.4.6 Migration from major 0

The v0→v1 translation is total and default-filling: every new field has a canonical default, so no value is unrecoverable — including the `CreateRegion` operation payload, whose embed-

ded `Region` default-fills structurally. This is unlike the *semantic*, context-dependent operation-payload migration of Section 6.7 (which could be irreversible); the major-1 migration needs no score context.

major-0 form	major-1 form
`Canvas` = `regions`	append `layout_defaults` = the A4/8 mm default (page 105 × 148.5, margins 7.5 staff spaces)
`Instrument` = `id`, `name`	append `range` = `None` (presence byte 0)
`Region` = ... (snapshot)	append `permits_spanning_slurs` = 0 (`false`)
`CreateRegion` op payload	default-fill the embedded `Region`'s appended field (`permits_spanning_slurs` = 0); *canonical* — a v0 op block migrates on read, and a block bearing a v1 `CreateRegion` is major 1 (a major-0-only reader opens the bundle read-only)
`ResolvedLayoutIR` `u64` prefixes	non-canonical `LayoutCache`: discard and regenerate at major 1 under `u32` (no in-place byte translation)
barrier / extension-declaration blobs	stay regime (b) `u64` (manifest-embedded, manifest stays major 0); unchanged in this bump
canonical-base `MaterializedState`	unchanged, byte-identical, stays major 0

A reader migrates a major-0 acceleration `Score` snapshot on read (cheaper than replaying the operation log); a writer emits only major-1 forms for the changed payloads and leaves the canonical base at major 0.

9.5 Schema Major 2

Schema major 2 is the second data-model expansion major: it fills the truncated bodies the v0 baseline deliberately staged to this companion — the cross-cutting structures (`Slur`, `Tie`, `Beam`, `Spanner`), `RepeatStructure`, `Staff` and `StaffLineConfiguration`, `Instrument`, and `ScoreMetadata` — to the shapes the core specification's Chapter 5 ratifies (schema-major-2 revision). The leaf types those shapes reference are defined in the same core revision; their wire layouts are below.

9.5.1 Where the changed fields reach

The major is assigned per payload type, exactly as in major 1:

- **Snapshot-only:** nothing, as of genesis tranche G1. `Instrument` was snapshot-only when this section was first ratified (no operation embedded one; it reached only the acceleration full-`Score` snapshot) and `RepeatStructure` likewise, until the repeat-authoring pair moved it to the canonical operation layer. `CreateInstrument` (genesis tranche G1, discriminant 31) moved `Instrument` the same way — see the next bullet, which now names it.
- **Canonical operation layer:** nine operation payloads embed filled values. `CreateInstrument` (genesis tranche G1 — Operation Catalog CREATEINSTRUMENT, discriminant 31) embeds the full `Instrument` value; its schema-major-2 appends

(`sound_config`, `default_clef`, `default_staff_lines`, and the rest) are mandatory fields, not `Option`-hidden, so it is *born at v2* exactly as `CreateRepeatStructure` below is — no lower-major layout for this payload exists, and every block carrying one stamps major 2 under minimal stamping. `CreateRepeatStructure` (Operation Catalog REPEAT STRUCTURES, discriminant 28) embeds the v2 `RepeatStructure` layout unconditionally (`kind` and `voltas` are not optional fields) and is likewise *born at v2*. Its sibling `DeleteRepeatStructure` (29) carries a bare identifier — a major-0 layout — so minimal stamping gives its blocks major 0; the kind discriminant itself is an append-only schema-*minor* vocabulary event (the same append mechanism the Phase-3 tranche used for 24–27 — a mechanism claim only: what a kind's blocks *stamp* always follows minimal stamping over its payload, as discriminant 24's always-major-2 `CreateStaff` shows). Neither changes anything about lower-major layouts. The seven remaining payloads of the first ratification embed filled values — `CreateCrossCutting` and `ModifyCrossCutting` (a `CrossCuttingValue` embedding `Slur`, `Tie`, `Beam`, or `Spanner`); `CreateStaff` (a `Staff`, which also embeds the filled `StaffLineConfiguration`); `CreateStaffInstance` (a `StaffInstance`, whose `staff_lines_override` embeds it); `SetStaffLayout` (whose *own* `staff_lines_override` field — an op-payload field, not part of a `StaffInstance` — embeds it); `SetMetadata` (a `ScoreMetadata`); and `CreateRegion` *transitively* — its embedded `Region` may carry `StaffInstance`s (`content.staff_instances`), each embedding the filled `StaffLineConfiguration`. (Reduction preconditions a *carried* region to be empty of typed children, but the byte form is representable and preserved in the canonical set, so the wire must define it.) That is 1 (`CreateInstrument`) + 1 (`CreateRepeatStructure`) + 7 = 9 payloads total.

Minimal stamping. A writer **MUST** stamp each operation-envelope block with the *lowest* schema major whose layouts decode the block's bytes to the encoded values. The stamp is therefore a pure function of the block's values — two conforming writers stamp identical content identically, which the content hash requires (the schema version is in the chunk-hash preimage). This is the same principle the major-1 section applies to the canonical base: byte-identical content **MUST NOT** be re-stamped at a higher major. A payload whose higher-major encoding coincides byte-for-byte with its lower-major form takes the lower major; a hidden embedding site (an `Option` that is `None`, an empty `Vec`) contributes nothing. Concretely, the per-payload minimal majors at this revision: `CreateCrossCutting` / `ModifyCrossCutting` / `CreateStaff` / `SetMetadata` ⇒ always **2** (their v2 fills are mandatory appended fields); `CreateRegion` ⇒ **2** iff a carried staff instance's `staff_lines_override` is `Some`, else **1** (the major-1 bool is always present); `CreateStaffInstance` / `SetStaffLayout` ⇒ **2** iff their `staff_lines_override` is `Some`, else **0**; `CreateRepeatStructure` ⇒ always **2** (born at v2); `DeleteRepeatStructure` ⇒ **0** (a bare identifier under a minor kind append); `TransposeInterval` ⇒ **0** (a sequence of identifiers and two `i32`s — every constituent is a major-0 layout, so the kind append is a schema *minor*, and a `TransposeInterval` rides in a major-0 block); every other payload keeps its prior major. A block stamps the maximum over its payloads. (Operation names are the Operation Catalog's; core Chapter 6's enum spells the creates `InsertStaff` / `InsertStaffInstance` / `InsertRegion`.)

- **The canonical base is unchanged.** The `MaterializedState` embeds none of the filled types (its values are effects, conflicts, anomalies, object states, pitch spellings, break keys, and pending records), so it remains major 0, byte-identical across the bump — the same keystone as major 1, and a conformance test **SHOULD** assert it again.

- The manifest stays major 0 (nothing here touches it).

9.5.2 Cross-major reader behaviour

The major-1 rules extend unchanged: non-canonical chunks at a foreign major are discarded and regenerated (or migrated on read, for the acceleration snapshot); canonical chunks parse or the bundle opens read-only. Migration *composes*: a major-2 reader migrates a major-0 snapshot v0→v1→v2 in one read, each step total and default-filling. A major-1-only reader opens a bundle read-only iff it carries an op block bearing a v2 value; a major-0-only reader additionally on v1 `CreateRegion` blocks, as before.

Staging note. As with major 1, the reference implementation raises chunk admission *per role* as the implementation phases of this revision's push land (snapshot decode, then the canonical operation layer); the accept-set stated in Section 9.1 is the ratified end-state, not a claim that every intermediate commit already admits it.

9.5.3 Changed value layouts

As in major 1, **the wire form ratifies the code, not the model's presentational field order**: a struct's v2 layout appends its new fields *after its existing major-0/major-1 fields*, even where Chapter 5's listing interleaves them. Concretely (every leaf under Chapter 5 regime (a) rules: LE integers, one discriminant byte per union, `u32` counts and length prefixes, one presence byte per `Option`, one bare byte per bool):

- `Slur` = `id` ‖ `start_event` ‖ `end_event` ‖ **`kind`** ‖ **`curvature_override`** ‖ **`style`**.
- `Tie` = (major-0 fields) ‖ **`style`**.
- `Beam` = `id` ‖ `events` ‖ `level` ‖ **`sub_beams`** ‖ **`geometry_override`**.
- `Spanner` = `id` ‖ `start` ‖ `end` ‖ `staves` ‖ **`kind`** ‖ **`style`** (the kind appends *after* `staves`; Chapter 5 lists it second, presentationally).
- `RepeatStructure` = `id` ‖ `start` ‖ `end` ‖ **`kind`** ‖ **`voltas`**.
- `Staff` = `id` ‖ `name` ‖ `abbreviation` ‖ `instrument` ‖ `default_staff_lines` ‖ `group` ‖ **`default_clef`** (appended last; Chapter 5 lists it mid-struct).
- `StaffLineConfiguration` = `line_count` ‖ **`line_spacing`** ‖ **`line_style`** ‖ **`bracket`**. This value is embedded by `Staff` *and* by `StaffInstance.staff_lines_override`, which is how `CreateStaffInstance`, `SetStaffLayout`, and (transitively, through a carried region's staff instances) `CreateRegion` become v2 payloads without gaining fields of their own (the `CreateRegion`-in-major-1 pattern).
- `Instrument` = `id` ‖ `name` ‖ `range` ‖ **`abbreviation`** ‖ **`sound_config`** ‖ **`transposition`** ‖ **`default_clef`** ‖ **`default_staff_lines`** ‖ **`unpitched_members`** (appended after the major-1 order `id/name/range`; Chapter 5's listing order is presentational).
- `ScoreMetadata` = `title` ‖ `composer` ‖ `copyright` ‖ **`subtitle`** ‖ **`lyricist`** ‖ **`arranger`** ‖ **`creation_timestamp`** ‖ **`modification_timestamp`** ‖ **`additional`** (appended after the major-0 order).

9.5.4 New leaf-type layouts

- `SlurKind`: one byte — 0 Legato, 1 Phrase, 2 Articulation, 3 Editorial.

- `CurveDirection`: one byte — 0 Above, 1 Below.
- `CurvatureOverride` = `direction` (`Option<CurveDirection>`) ‖ `height` (`Option<SpaceUnit>`).
- `LineStyle`: one byte — 0 Solid, 1 Dashed, 2 Dotted.
- `SpanStyle` (one shared record: `Slur.style`, `Tie.style`, `Spanner.style`) = `line` (`LineStyle`) ‖ `thickness` (`Option<SpaceUnit>`).
- `SubBeam` = `level` (one byte) ‖ `events` (`u32` count ‖ `EventId` *leaves*, each `u32`-length-framed 16 canonical bytes = 20 bytes — this chapter's leaf framing, exactly as `Beam.events` encodes today).
- `BeamGeometryOverride` = `slope` (`Option<CanonicalF64>`; dimensionless) ‖ `offset` (`Option<SpaceUnit>`).
- `SpannerKind`: discriminant byte ‖ payload — 0 Generic; 1 Hairpin ‖ `HairpinDirection` (one byte: 0 Crescendo, 1 Diminuendo); 2 OctaveLine ‖ `OctaveOffset` (one `i8`); 3 PedalLine ‖ `PedalKind` (one byte: 0 Sustain, 1 Sostenuto, 2 UnaCorda); 4 TrillExtension; 5 Glissando; 6 Portamento; 7 TextLine ‖ `TextLineDefinition` (= one length-prefixed raw-UTF-8 string); 8 Bracket ‖ `BracketKind` (one byte: 0 Square).
- `RepeatKind`: discriminant byte ‖ payload — 0 SimpleRepeat ‖ `count` (`u32` LE); 1 DaCapo ‖ `end_target` (`TimeAnchor`); 2 DalSegno ‖ `segno` ‖ `end_target`; 3 Volta.
- `Volta` = `endings` (`u32` count ‖ bare `u32` LE values — fixed-width primitives are unframed) ‖ `start` ‖ `end` (`TimeAnchor`s).
- `SpaceUnit` = one `CanonicalF64`, leaf-framed (12 bytes) — byte-identical to how `CanonicalF64` itself encodes; the newtype adds no bytes.
- `StaffBracketKind`: one byte — 0 Brace, 1 Bracket.
- `SoundConfiguration` = `u32` length ‖ raw bytes (opaque; never interpreted by the core).
- `TranspositionInterval` = `diatonic_steps` (`i32` LE) ‖ `chromatic_steps` (`i32` LE).
- `UnpitchedMember` = `member` (bare `u32` LE — `UnpitchedMemberId` is an instrument-scoped small value, *not* a 128-bit object identifier, so the identifier-leaf framing does not apply; this matches how `UnpitchedEvent.instrument_member` has encoded since major 0) ‖ `name` (string) ‖ `staff_position` (bare `i16` LE, likewise).
- `Timestamp` = one `i64` LE (nanoseconds since the Unix epoch, UTC).
- `MetadataEntry` = `key` (string) ‖ `value`; `MetadataValue`: discriminant byte ‖ payload — 0 Text ‖ string; 1 Integer ‖ `i64` LE; 2 Flag ‖ bool.
- `Clef`, `ClefShape`, `KeySignature`, `ClefChange`, `KeySignatureChange` are *not new wire forms*: they have carried major-0 layouts since the visible-slice revision (empty sequences left `Score` bytes unchanged). Major 2 only *embeds* `Clef` in two more places (`Staff.default_clef`, `Instrument.default_clef`). Its layout, for the record: `shape` (one byte: 0 G, 1 F, 2 C, 3 Percussion) ‖ `line` (`i8`) ‖ `octave_shift` (`i8`); `KeySignature` = `fifths` (`i8`; decoders **MUST** reject $|\texttt{fifths}| > 7$).

As in major 1: every embedded *leaf* keeps this chapter's framing (an identifier is a `u32` LE length ‖ its 16 canonical bytes = 20; a `CanonicalF64` is a `u32` LE length ‖ 8 bytes = 12); fixed-width primitive integers and bools inside structs are bare; every `Option` is one presence byte then the payload when present.

9.5.5 Migration from major 1

Total and default-filling, needing no score context; composes after the major-1 migration for major-0 input. The defaults reproduce today's rendering exactly: a migrated document draws byte-identically.

major-1 form	major-2 form
`Slur`	append `kind` = Legato, `curvature_override` = `None`, `style` = (Solid, `None`)
`Tie`	append `style` = (Solid, `None`)
`Beam`	append `sub_beams` = empty, `geometry_override` = `None`
`Spanner`	append `kind` = Generic, `style` = (Solid, `None`)
`RepeatStructure`	append `kind` = SimpleRepeat {count = 2} (a v1 repeat *meant* a repeat), `voltas` = empty
`Staff`	append `default_clef` = treble (G, line 2, shift 0)
`StaffLineConfiguration`	append `line_spacing` = 1.0, `line_style` = Solid, `bracket` = `None` — at *every* embedding site: `Staff.default_staff_lines`, `Instrument.default_staff_lines` (v2-new), `StaffInstance.staff_lines_override`, and the `SetStaffLayout` op payload's own `staff_lines_override` field
`Instrument`	append `abbreviation` = `None`, `sound_config` = empty, `transposition` = `None`, `default_clef` = treble, `default_staff_lines` = the complete v2 default (`line_count` = 5, `line_spacing` = 1.0, `line_style` = Solid, `bracket` = `None`), `unpitched_members` = empty
`ScoreMetadata`	append `subtitle` / `lyricist` / `arranger` = `None`, both timestamps = 0 (the unset convention), `additional` = empty
the seven embedding op payloads (incl. `CreateRegion`'s transitively embedded staff instances)	default-fill the embedded value(s) as above; *canonical* — a v0/v1 op block migrates on read, and a block bearing any v2-dependent bytes is major 2 (a lower-major-only reader opens the bundle read-only)
canonical-base `MaterializedState`	unchanged, byte-identical, stays major 0

9.6 Schema Major 3

Schema major 3 is the third data-model expansion major (Push 4b tranche 3b-i, SCORE TUNING CONTEXT): `ScoreTuningContext` gains `smufl` (the score's declared SMuFL version requirement) and `overrides` (per-scope tuning overrides) on the canonical wire. `accidental_extensions` — the third field the core specification added alongside these two — is deliberately **staged** to a later major: its consumer, the engraver, does not exist yet, and freezing a wire layout ahead of a real consumer is exactly what the frozen-layout rule (Requirement 5.3) warns against. When

`accidental_extensions` does land, it appends *after* `overrides`, per the append-only rule this chapter has followed since major 1.

9.6.1 Where the changed fields reach

As first defined (Push 4b tranche 3b-i), no operation payload embedded the tuning context — `ScoreTuningContext` reached the wire only through the acceleration full-`Score` snapshot, and the `OperationEnvelopeBlock` role's accept-set stayed at major 2. This changed at genesis tranche G2b (`spec/CONTRACT_GENESIS_G2B_TUNING.md`), which corrects the claim below:

- **Reaches two sites, as of genesis tranche G2b.** `ScoreTuningContext` still reaches the wire through the acceleration full-`Score` snapshot, *and* its authored subset — `TuningContextSettings`, exactly the five wire-bearing fields, byte-identical to this section's layout (Section 6.3, 34 `SetTuningContext`) — now reaches the wire through the canonical operation layer: `SetTuningContext` is *born at v3 unconditionally* (the same shape as `CreateInstrument`'s v2 birth, Section 9.5), since `smufl` and `overrides` are mandatory appends with no lower-major layout for this payload to fall back to. The `OperationEnvelopeBlock` role's accept-set therefore **widens to major 3** (Section 9.1) — schema major 3 stopped being the data-model bump under which a chunk role's max does not move in lockstep with the others; that distinction now belongs to no major.
- **The canonical base is unchanged.** The `MaterializedState` embeds no tuning context (nor any other `Score` field value), so it remains major 0, byte-identical across the bump — the same keystone as majors 1 and 2, and a conformance test **SHOULD** assert it again. This is unaffected by G2b: the canonical carrier of the tuning context is the operation log, not the base, so the base's own stamping never moves.
- The manifest stays major 0 (nothing here touches it).

9.6.2 Cross-major reader behaviour

The major-1/major-2 rules extend unchanged: the acceleration snapshot is non-canonical, so a reader **MAY** discard and regenerate a foreign-major one, or migrate it on read (the reference implementation migrates). Canonical chunks parse or the bundle opens read-only. Migration *composes*: a major-3 reader migrates a major-0 snapshot v0→v1→v2→v3 in one read, each step total and default-filling. As originally defined (Push 4b tranche 3b-i), no operation payload changed at this major, so a major-2-only reader's op-block admission was exactly as before the bump and only the `Snapshot` role's accept-set widened, to 3 (Section 9.1). Genesis tranche G2b (above) changes this: a major-2-only reader now meets a possible major-3 op block (`SetTuningContext`) and opens read-only, exactly the one-way door Section 9.1 describes for every other accept-set raise.

9.6.3 Changed value layout

As in majors 1 and 2, the wire form appends new fields *after* the existing ones, independent of the core specification's presentational field order (Section 9.4's note applies unchanged):

- `ScoreTuningContext` = `default_pitch_space` ‖ `default_tuning_system` ‖ `reference` (the frozen major-0/1/2 prefix, unchanged since major 0) ‖ **`smufl`** ‖ **`overrides`**. `accidental_extensions` is **not** on the wire this major (see above): a

decoder default-fills it, exactly as it always has, and a future major appends it after `overrides` when it lands.

9.6.4 New leaf-type layouts

- `SmuflVersion` = `major` (`u16` LE, bare) ‖ `minor_centi` (`u16` LE, bare) — the fractional SMuFL version digits normalized to hundredths (1.12 → 12, 1.4 → 40, 1.3 → 30) so an integer comparison on the pair agrees with SMuFL's real release order (1.12 shipped before 1.3, though 3 literally sorts before 12).
- `SmuflVersionRequirement` = `minimum` (`SmuflVersion`) ‖ `authored_against` (`SmuflVersion`) — 8 bytes total, neither field framed as a length-prefixed leaf (both are fixed-shape bare-integer pairs).
- `TuningScope`: one discriminant byte ‖ payload — 0 Voice ‖ `VoiceId`; 1 Staff ‖ `StaffId`; 2 Region ‖ `RegionId`; 3 Range ‖ `start` (`TimeAnchor`) ‖ `end` (`TimeAnchor`) ‖ `voices` (`VoiceSelector`), fields in declaration order.
- `TuningOverride` = `scope` (`TuningScope`) ‖ `pitch_space` (`Option<PitchSpaceId>`) ‖ `tuning_system` (`Option<TuningSystemId>`) ‖ `reference` (`Option<ReferencePitch>`) — one presence byte per `Option`, the inner catalog-id/pitch leaves under this chapter's regime (a) framing (a catalog id is a length-prefixed string; `ReferencePitch` carries its own established layout).

9.6.5 Migration from major 2

Total and default-filling, needing no score context; composes after the major-1 and major-2 migrations for major-1/major-0 input. The flat 12 default bytes this migration appends (`smufl` = 8, `overrides`' empty count = 4) are the same for every score, independent of its other content, since neither field depends on anything already on the wire.

major-2 form	major-3 form
`ScoreTuningContext`	append `smufl` = 1.4/1.4 (both `minimum` and `authored_against`, the version this reference implementation already targets), `overrides` = empty
canonical-base `MaterializedState`	unchanged, byte-identical, stays major 0

9.7 Schema Major 4

Schema major 4 is the fourth data-model expansion major (X3c): a `Tuplet` gains `display` (core specification TUPLETS AS GROUPING OBJECTS, `req:time:tuplet-display`), whether its number and bracket are shown. The core specification has listed the field since its first revision; no earlier major carried it, so a tuplet a file marks hidden had nowhere in the score to land, and its marks were drawn regardless.

9.7.1 Where the changed fields reach

- **Two sites.** `Tuplet` reaches the wire through the acceleration full-`Score` snapshot (the cross-cutting registry's `tuplets`), and through the canonical operation layer: `CreateTuplet` (40) carries the whole value, so it is *born at v4 unconditionally*, as `SetTuningContext` was born at v3, `display` being a mandatory append with no lower-major layout for the payload to fall back to. The `OperationEnvelopeBlock` and `Snapshot` roles' accept-sets therefore both **widen to major 4** (Section 9.1).
- **The canonical base is unchanged.** The `MaterializedState` embeds no tuplet (nor any other `Score` field value), and no reduction verdict reads a display, so it remains major 0, byte-identical across the bump, as for majors 1 to 3.
- The manifest stays major 0.

9.7.2 Cross-major reader behaviour

The rules of the earlier majors extend unchanged for the acceleration snapshot: a major-4 reader migrates a snapshot of any lower major in one read, v0→v1→v2→v3→v4, each step total and default-filling. A major-3-only reader meeting a major-4 op block opens read-only, the one-way door Section 9.1 describes.

A `CreateTuplet` written before this major carries the five-field `Tuplet` of majors 0 to 3. Its operation envelope is canonical and its bytes are its identity, so it cannot be migrated in place: a migrated envelope would be another operation. Before the product's 1.0 release no such history is supported. A conforming decoder **MUST** reject such an envelope, since the five-field value is no canonical major-4 `Tuplet`; the reference implementation recognizes the older layout and refuses it by name (`EnvelopeDecodeError::UnsupportedLayout`), so it reads as a version boundary and not as corruption. The decode corpus pins the rejection (`create_tuplet_before_major_4`).

9.7.3 Changed value layout

As in majors 1 to 3, the wire form appends the new field *after* the existing ones, independent of the core specification's presentational field order (Section 9.4's note applies unchanged):

- `Tuplet` = `id` ‖ `ratio` ‖ `members` ‖ `parent` ‖ `required_total` (the frozen major-0 to major-3 prefix) ‖ **`display`**.

9.7.4 New leaf-type layouts

- `TupletNumber`: one discriminant byte — 0 Actual, 1 None. Growth is by appended discriminant.
- `TupletBracket`: one discriminant byte — 0 Auto, 1 Hidden. Growth is by appended discriminant.
- `TupletDisplay` = `number` (`TupletNumber`) ‖ `bracket` (`TupletBracket`) — two bytes, neither length-framed.

9.7.5 Migration from major 3

Total and default-filling, needing no score context; composes after the earlier migrations for lower-major input. Each tuplet gains two zero bytes, the default display.

major-3 form	major-4 form
`Tuplet`	append `display` = `Actual` / `Auto`, the number and bracket every tuplet was drawn with before this major
canonical-base `MaterializedState`	unchanged, byte-identical, stays major 0

9.8 Schema Major 5

Schema major 5 is the fifth data-model expansion major (X4b, roadmap D58): expression and text. An event's mark, ornament and grace slots take their real types; a `Marker` gains its kind; the lyric line becomes a `Lyric`, one syllable; text is held to Unicode NFC; `LineStyle` and `SpannerKind` gain a variant each; and the score gains a voice's home staff.

9.8.1 Where the changed fields reach

- **The filled slots.** `PitchedEvent` and `UnpitchedEvent` already carried `articulations`, `ornaments` (a note's) and `grace`, each empty in every value any writer produced. Their types change, and their names: `articulations` is `marks`. An empty slot encodes byte-identically to the placeholder it replaces (a zero count, an absent option), so an `InsertEvent` or `ModifyEvent` written before this major decodes unchanged, and one whose value holds a mark, an ornament or a grace is *born at v5*; any other is stamped as before. No field is appended to a value an operation carried before this major.
- **Cross-cutting values.** `CrossCuttingValue` appends 4 Marker and 5 Lyric; a create or modify carrying either, a wavy line or a pedal line without its sign is born at v5, any other at v2 as before. The staff, instrument, staff-instance, layout and region payloads are born at v5 where a staff-line style they carry is wavy.
- **The snapshot.** `Marker` appends `kind`, `Lyric` replaces `LyricLine` in the cross-cutting registry's `lyrics`, and `Score` appends `voice_homes`. No operation created a marker or a lyric line before this major, so neither layout change reaches an operation written earlier. The `OperationEnvelopeBlock` and `Snapshot` roles' accept-sets both **widen to major 5** (Section 9.1).
- **The canonical base is unchanged.** The `MaterializedState` embeds no `Score` field value; a voice's home and the new values' write registers are reducer state, rebuilt from the operations, so it remains major 0, byte-identical across the bump.
- The manifest stays major 0.

9.8.2 Cross-major reader behaviour

A major-5 reader reads a snapshot of any lower major through that major's frozen layout, the event read as major 4 wrote it. A value no major before 5 gave a meaning, and no writer before it produced, **MUST** be rejected rather than given a default: a placeholder list with an element,

a grace kind, a marker (it had no kind), a lyric line (it had no syllable), a wavy line and a pedal line without its sign. The reference implementation refuses each by name; the decode corpus pins each refusal under `core.score_v4` (class `major-5-value`). The older majors' decoders read `LineStyle` and `SpannerKind` with the live codec, which knows the appended tags, so a reader holds those two by checking the decoded score. A major-4-only reader meeting a major-5 op block opens read-only, the one-way door Section 9.1 describes.

9.8.3 Changed value layout

The wire form appends new fields *after* the existing ones, and a filled slot keeps its place:

- `PitchedEvent` = ... ‖ `pitches` ‖ **`marks`** (seq⇑(`EventMark`)) ‖ `dynamic` ‖ **`ornaments`** (seq⇑(`Ornament`)) ‖ `stem` ‖ **`grace`** (`Option<Grace>`); `UnpitchedEvent` likewise without `ornaments`. Marks and ornaments are canonical sets: at most one of each kind, ascending by kind (a mark's discriminant, an ornament's `kind`), a repeated kind or an unordered pair rejected (class `marks-out-of-order`). `dynamic` stays the reserved unit `DynamicMark`: a dynamic is a marker, so the slot is absent, and a present one is rejected by name at every major (class `dynamic-placeholder`, under `core.score_v4` and `core.score_v5`).
- `Marker` = `id` ‖ `anchor` ‖ **`kind`** (`MarkerKind`).
- `Lyric` = `id` (`LyricLineId`) ‖ `event` (`EventId`) ‖ `verse` (`u16` LE) ‖ `text` (`Text`) ‖ `syllabic` (`Syllabic`) ‖ `extension` (bool), replacing `LyricLine` = `id` ‖ `events`.
- `Score` = ... (the major-4 fields) ‖ **`voice_homes`** (`u32` count ‖ entries ascending by `VoiceId`, each a `VoiceId` leaf ‖ a `StaffId` leaf, 20 bytes each in this chapter's leaf framing).
- `TextLineDefinition` = `text` (`Text`), the same bytes as before, now held to NFC.

9.8.4 New leaf-type layouts

- `Text`: a length-prefixed UTF-8 string, rejected at decode unless it is in Unicode Normalization Form C.
- `EventMark`: discriminant byte ‖ payload — 0 Staccato, 1 Staccatissimo, 2 Spiccato, 3 Tenuto, 4 DetachedLegato, 5 Accent, 6 Marcato, 7 Stress, 8 Unstress, 9 UpBow, 10 DownBow, 11 Harmonic, 12 OpenString, 13 Stopped, 14 SnapPizzicato, 15 Scoop, 16 Plop, 17 Doit, 18 Falloff; 19 Tremolo ‖ `strokes` (`u8`); 20 TremoloWithNext ‖ `strokes` (`u8`); 21 Arpeggio ‖ `ArpeggioDirection` (one byte: 0 Plain, 1 Up, 2 Down).
- `Ornament` = `kind` (`OrnamentKind`, one byte: 0 Trill, 1 Mordent, 2 InvertedMordent, 3 Turn, 4 InvertedTurn) ‖ `accidental_above` (`Option<AccidentalId>`) ‖ `accidental_below` (`Option<AccidentalId>`).
- `Grace` = `kind` (`GraceKind`, one byte: 0 Acciaccatura, 1 Appoggiatura; major 0's 2 and 3, which nothing wrote, are gone) ‖ `value` (`NoteValue`) ‖ `dots` (`u8`) ‖ `order` (`u16` LE).
- `MarkerKind`: discriminant byte ‖ payload — 0 Dynamic ‖ `Dynamic`; 1 Fermata ‖ `Fermata`; 2 Breath ‖ `BreathMark` (one byte: 0 Comma, 1 Tick); 3 Caesura ‖ `CaesuraMark` (one byte: 0 Normal, 1 Thick, 2 Short, 3 Curved); 4 Text ‖ `Text`; 5 Tempo ‖ `TempoMark`; 6 Rehearsal ‖ `Text`; 7 Segno; 8 Coda.
- `Dynamic`: discriminant byte — 0–13 the marks from *pppppp* to *ffffff*, 14 fp, 15 sf, 16 sfz, 17

sffz, 18 sfp, 19 sfpp, 20 rf, 21 rfz, 22 fz, 23 niente; 24 Other ‖ `Text`.

- `Fermata` = `shape` (`FermataShape`, one byte: 0 Normal, 1 Short, 2 Long, 3 VeryShort, 4 VeryLong) ‖ `inverted` (bool).
- `TempoMark` = `text` (`Option<Text>`) ‖ `metronome` (`Option<Metronome>`); `Metronome` = `beat` (`NoteValue`) ‖ `dots` (`u8`) ‖ `per_minute` (`Text`).
- `Syllabic`: one byte — 0 Single, 1 Begin, 2 Middle, 3 End.

9.8.5 Appended variants

`LineStyle` appends 3 Wavy, and `SpannerKind` appends 9 PedalBracket ‖ `PedalKind`, a pedal line drawn without its sign; 3 PedalLine is unchanged and is drawn with its sign. Neither tag existed before this major, so no earlier document holds either, and a value holding one is stamped major 5 rather than at a raised minor of an earlier major.

9.8.6 Migration from major 4

Total for every value a major-4 writer produced, needing no score context.

major-4 form	major-5 form
an event's empty slots	the same bytes, read as empty `marks`, `ornaments` and no `grace`
a filled slot, a marker, a lyric line	rejected by name; no writer produced one
`Score`	append `voice_homes` = empty (four zero bytes)
canonical-base `MaterializedState`	unchanged, byte-identical, stays major 0

Chapter 10

Non-Canonical Pinned Encodings

Non-canonical means: never part of document identity. Nothing in this chapter enters a content hash that canonical state depends on, and all of it can be discarded and rebuilt without changing the document. The encodings are pinned for *reproducibility* — byte-equal conformance claims and cache correctness — not for durability.

10.1 `LayoutObjectId` (`MUSCLOID`)

Governed by core requirement `req:layoutir:object-id-derivation`. `LayoutObjectId` = trunc128(BLAKE3(`MUSCLOID` ‖ *inputs*)), with three preimage shapes:

stable *inputs* = the source `TypedObjectId`'s canonical bytes.

manifestation *inputs* = source canonical bytes ‖ `RegionId` canonical bytes (16 BE) — distinct regions give one source distinct manifestation ids.

synthesized *inputs* = source canonical bytes ‖ five `u64` LE words: the `SynthesisKind` discriminant, the registry id's high and low 64 bits (zero unless `Registered`), and the instance key's high and low 64 bits.

`SynthesisKind` discriminants: 0 CancellationAccidental, 1 KeySignatureNatural, 2 GeneratedRest, 3 EngravedBreak, 4 MultimeasureRest, 5 Cautionary, 6 Registered.

10.2 `ResolvedLayoutIR` Canonical Output

The resolved layout's canonical bytes are the **byte-equal conformance surface** of core Chapter 7: two conforming engravers given the same score, profile, and glyph catalog must produce identical bytes. In schema major 0 the encoding is regime (b) (`u64` LE counts and length prefixes); schema major 1 moves it to regime (a)'s `u32` prefixes with the rest of the unification (Section 9.4), re-locking this surface's goldens. Because the surface is non-canonical, a reader regenerates a foreign-major layout cache rather than decoding it. Either way every geometric coordinate is quantized to the 1/1024 staff-space grid as a `QuantizedCoord` (8 LE bytes; Chapter 4); a non-finite or out-of-range coordinate is a determinism violation and **MUST** be rejected, never normalized. The top-level section order is: source score version ‖ pages ‖

glyphs ‖ strokes ‖ engraving decisions ‖ glyph-catalog identity; within each section, elements carry their provenance (built on `MUSCLOID` ids) and their quantized geometry. The full leaf grammar is pinned by the reference implementation's golden and round-trip anchors (Chapter 11) rather than restated field-by-field here, because the surface is non-canonical and evolves with the engraving feature set.

10.3 Layout Caches and Font Metrics

`LayoutCache` chunks (`ChunkKind` 6) hold cached layout artifacts; they are **always discardable**, and a reader **MUST NOT** treat a missing, stale, or undecodable layout cache as bundle corruption. The glyph-metrics identity hash is domain-separated under `MUSCFNTM` (a reserved built-in, non-canonical tag): it names the metrics a layout was computed against, so caches and conformance runs can detect font drift; it never enters canonical state.

Chapter 11

Golden Anchor Registry

The conformance contract binding this document to the reference implementation. Three lock classes:

literal-byte the test asserts exact expected bytes or exact discriminant literals — the strongest lock;

round-trip the test asserts decode ∘ encode = id and re-encode byte-stability over a corpus;

canonical-equality the test asserts that semantically equal values encode identically (order-independence, reduction identities).

The first block restates the anchors already recorded in the core specification's `app:bytes` Reference-implementation locks table (which governs them); the second block adds the anchors this document introduces, which `app:bytes` predates.

Each anchor names its crate file, then the test on a second line.

Layout	Anchor	Class
Imported from `app:bytes`:		
`TypedObjectId`	`epiphany-core/src/ids.rs` `::typed_object_id_byte_form_is_locked`	literal-byte
Promoted `VoiceId`	`epiphany-core/src/graph.rs` `::promoted_voice_id_byte_form_is_locked`	literal-byte
System `PitchId`	`epiphany-core/src/pitch.rs` `::system_pitch_id_byte_form_is_locked`	literal-byte
`IntegrityAnomalyId`	`epiphany-ops/src/anomaly.rs` `::integrity_anomaly_id_byte_form_is_locked`	literal-byte
`ObjectKind` (ops)	`epiphany-ops/src/support.rs` `::object_kind_discriminants_are_golden`	literal-byte
`ChunkKind`	`epiphany-bundle/src/chunk.rs` `::chunk_kind_discriminants_are_golden`	literal-byte
`CompressionAlgorithm`	`epiphany-bundle/src/chunk.rs` `::compression_algorithm_encoding_is_golden`	literal-byte
`ProfileId`	`epiphany-bundle/src/superblock.rs` `::profile_id_discriminants_are_golden`	literal-byte
`ManifestId`	`epiphany-bundle/src/ids.rs` `::manifest_id_is_content_derived_and_deterministic`	canonical-equality
`TransactionCategory`	`epiphany-ops/src/payload.rs` `::transaction_category_discriminants_are_golden`	literal-byte

Layout	Anchor	Class
`ResolutionAction`	`epiphany-ops/src/conflict.rs ::resolution_action_discriminants_are_golden`	literal-byte
`BlobId`	`epiphany-determinism/src/hash.rs` `ContentHash::of_blob` tests	round-trip
`RationalTime`	`epiphany-core/src/time.rs ::equal_rationals_encode_identically`	canonical-equality
`MUSCLOID` derivation	`epiphany-layout-ir/src/provenance.rs ::stable_id_uses_the_ratified_muscloid_derivation`	literal-byte
Domain-tag spellings	`epiphany-determinism/src/domain.rs ::exact_tag_spellings_match_spec`	literal-byte
Added by this document:		
`OperationKind` discriminants (§6.3)	`epiphany-ops/src/payload.rs ::operation_kind_wire_discriminants_are_golden`	literal-byte
`OperationPayload` discriminants (§6.2)	`epiphany-ops/src/payload.rs ::operation_payload_discriminants_are_golden`	literal-byte
`OperationKindTag` (§6.4)	`epiphany-ops/src/payload.rs ::operation_kind_tag_decode_mirrors_encode_exactly`	round-trip
`ResolveEquivocation` payload (§6.2)	`epiphany-ops/src/payload.rs ::resolve_equivocation_payload_encodes_target_then_hash`	literal-byte
`OperationStamp` (§6.5)	`epiphany-ops/src/stamp.rs ::stamp_encode_is_stable`	literal-byte
`CausalContext` (§6.5)	`epiphany-ops/src/causal.rs ::canonical_encoding_is_build_order_independent`	canonical-equality
`MaterializedState` (§6.6.1)	`epiphany-ops/src/decode.rs ::reduced_states_decode_and_reencode`	round-trip
Operation-index payload (§7.6)	`epiphany-bundle/src/opindex.rs ::payload_encoding_is_golden`	literal-byte
Fixed header (§7.1)	`epiphany-bundle/src/header.rs ::header_round_trips`	round-trip
Superblock (§7.2)	`epiphany-bundle/src/superblock.rs ::superblock_round_trips_through_256_bytes`	round-trip
Chunk hash preimage (§7.3)	`epiphany-bundle/src/chunk.rs ::hash_preimage_matches_the_spec_layout`	literal-byte
`ChunkRef` (§7.3.1)	`epiphany-bundle/src/chunk.rs ::chunk_ref_round_trips`	round-trip
Manifest body (§7.5)	`epiphany-bundle/src/manifest.rs ::re_encode_is_byte_identical`, `::manifest_round_trips`, `::duplicate_roots_collapse_on_encode`, `::semver_orders_numerically_not_byte_wise`	round-trip + canonical-equality
Barrier kinds blob (§8.1)	`epiphany-layout-ir/src/barrier.rs ::affected_object_kinds_blob_bytes_are_golden`	literal-byte
Barriers blob (§8.2)	`epiphany-layout-ir/src/barrier.rs ::edit_barriers_blob_bytes_are_golden`	literal-byte
Barrier variants (§8.2)	`epiphany-layout-ir/src/barrier.rs ::every_scope_and_condition_variant_round_trips_byte_identically`	round-trip
Whole-`Score` codec (§5.1)	`epiphany-core/src/codec.rs ::generator_scores_round_trip`, `::trailing_and_truncated_bytes_are_rejected`	round-trip

Layout	Anchor	Class
`CanonicalValue` seam (§5.5)	`epiphany-core/src/codec.rs ::value_types_round_trip_over_generator_ corpus`	round-trip

Struct *bodies* in Chapter 5 are round-trip-locked rather than literal-byte-locked: their byte identity follows from the frozen positional rule plus the literal-byte locks on every discriminant and leaf they embed.

11.1 The Decode Vector Corpus

The cross-implementation decoder test is **delivered**, as a corpus of literal byte strings each paired with the verdict a decoder owes it. The reference implementation carries it at `spec/vectors/decode_vectors.txt` and regenerates it from a committed generator, so a change to any wire form moves the file.

Each line is `surface verdict class name hex`. The `verdict` is `accept` or `reject`. The `class` names *why* a rejected vector is rejected and is **informative only**: implementations need not agree on an error taxonomy.

REQUIREMENT 11.1

A conforming decoder **MUST** accept every `accept` vector for a surface it implements, and **MUST** reject every `reject` vector for that surface. A value decoded from an `accept` vector **MUST** re-encode to exactly that vector's bytes.
Accepting a `reject` vector and then normalizing it is *accepting* it, and does not satisfy this requirement. Canonical decode is injective: distinct byte strings denote distinct values, which is what content-addressing rests on.

RATIONALE

The corpus is not a restatement of this document; it is what a second implementation can be run against. Every rejection class in it is one the reference implementation shipped a bug in, or one whose check no injectivity fuzzer can see: a whole-value re-encode guard catches the fields a decoder *normalizes* and is blind to order-preserving sequence fields, which need per-site order checks; and a guard on an outer value can *mask* a lenient inner codec rather than fix it (Push 5).
A future revision should extend the corpus to the representative struct layouts of Section 5.4, which remain round-trip locked rather than literal-byte locked.
Delivered (`spec/CONTRACT_CORE_DECODE_VECTORS.md`). The corpus now additionally pins all five representative layouts of Section 5.4 (`RationalTime`, `TimeAnchor`, `Pitch`, `Event`, `Slur`), the schema-major-3 tuning-context leaves (`SmuflVersion`, `SmuflVersionRequirement`, `TuningScope`, `TuningOverride`) and their container (`ScoreTuningContext`), and one whole-`Score` snapshot at each schema major `0`–`3`. The mandatory regression vector is the one that motivated this extension: bytes encoding `ScoreTuningContext` with `overrides` before `smufl` — the exact self-consistent field

swap that previously passed the entire reference-implementation suite and 8/8 conformance, because round-trip locking cannot see a decoder and encoder agreeing on the wrong order. Each per-major whole-`Score` vector genuinely decodes at its stamped major through `Score::decode_canonical_versioned`; majors `0`–`2` assert only that decoding succeeds (a migration deliberately rewrites bytes to the current layout, so the injectivity check there is that the input was already canonical *at its own major*, which the frozen per-major encoder's own re-encode guard already established), while major `3` additionally asserts the decoded value re-encodes to exactly the input bytes.

Chapter 12

Revision History

Date	Section	Change
October 11, 2026	All	0.1.0 — Initial companion: ratifies the previously provisional whole-`Score` value codec (core P11-4), operation-layer wire forms (ops), bundle physical layout (bundle P11-D2/D4/D5), operation-index payload (P12-D1), and extension-blob/edit-barrier byte forms (P12-E1/E2/E3); pins the no-varint rule, the frozen-positional schema-evolution keystone, and the id-leads envelope property; `SnapshotId` derivation deferred (open question).
October 11, 2026	Operation wire forms	0.2.0 — Phase-3 first tranche: appended `OperationKind` wire discriminants 24–27 (`CreateStaff`, `SetTimeSignature`, `SetTempoSegment`, `SetStaffLayout`) with their payload layouts, the matching `OperationKindTag` discriminants 24–27 (kind-to-tag naming gains `CreateStaff` → `InsertStaff`), and `PreconditionFailureReason` 11 (`TempoMapMalformed`) — a schema-*minor* evolution under this document's own append-only rules (Chapter 9); no existing assignment changed. Semantics: Operation Catalog 0.5.0 (CreateStaff, Meter and Tempo Overwrites, SetStaffLayout, and the value-restoring UndoTransaction revision).

Date	Section	Change
October 11, 2026	Schema evolution / Graph value layouts	0.3.0 — Defines **schema major 1**, the first data-model expansion major (Section 9.4): appends `Canvas.layout_defaults` (new `CanvasLayoutDefaults`/`CanvasSize`/`CanvasMargins` layouts, snapshot-only), `Instrument.range` (`Option<PitchRange>`, snapshot-only), and `Region.permits_spanning_slurs` (also inside the *canonical* `CreateRegion` operation payload); narrows regime (b) by unifying the non-canonical resolved-layout length prefixes to `u32` (the manifest-embedded barrier blobs stay `u64`, partly resolving the standing open question); pins per-payload-type major assignment (the canonical-base `MaterializedState` stays major 0, byte-identical), the accept-set gate ([MIN, MAX]), the cross-major reader rules (discard-and-regenerate non-canonical chunks; parse-or-read-only for canonical ones, so a major-0 reader opens a bundle with v1 `CreateRegion` ops read-only), and the total default-filling v0→v1 migration table. Clarifies that any field add is a major change regardless of `Option`-ness.
October 11, 2026	Effect vocabularies / Extension blobs	0.4.0 — Pass-12 G-ratification: appends `PreconditionFailureReason` 12 (`SystemDerivedContentImmutable`, P12-K3) and 13 (`RecreateContentMismatch`, P12-K9), and `ReanchorReason` 6 (`SameCanvasNearer`, P12-C4) — schema-*minor* appends under Chapter 9; no existing assignment changed. Adds the unsafe-edit tombstone *encoding* open question (from P12-E5; semantics ratified in core Chapter 8). Semantics: Operation Catalog 0.6.0.

Date	Section	Change
October 11, 2026	Schema evolution / Graph value layouts	0.5.0 — Defines **schema major 2**, the second data-model expansion major (Section 9.5): fills the staged truncations — `Slur` (kind / curvature override / style), `Tie` (style), `Beam` (sub-beams / geometry override), `Spanner` (kind, with the leading `Generic` migration default / style), `RepeatStructure` (kind / voltas), `Staff` (`default_clef`), `StaffLineConfiguration` (spacing / line style / bracket), `Instrument` (six appended fields), and `ScoreMetadata` (six appended fields incl. the strictly-authored timestamps) — with layouts for the new leaf types and the total default-filling v1→v2 migration table. Assigns the major per payload type: snapshot-only for `Instrument` / `RepeatStructure`; *canonical* for the seven embedding op payloads (`Create/ModifyCrossCutting`, `CreateStaff`, `CreateStaffInstance`, `SetStaffLayout`, `SetMetadata`, and `CreateRegion` transitively through carried staff instances); the canonical base and manifest stay major 0, byte-identical. Accept-set widens to $\{0, 1, 2\}$ (staged per role as the implementation phases land). Semantics: core specification schema-major-2 revision (leaf-type definitions, clef/key content-model ratification, `ClefId` retirement, strictly-authored timestamp requirement).
October 11, 2026	Operation vocabularies	0.6.0 — Repeat authoring (schema-major-2 revision, Phase D): appends `OperationKind` 28 (`CreateRepeatStructure`, lp(`RepeatStructure`), born at v2 — every block carrying one stamps major 2 under minimal stamping) and 29 (`DeleteRepeatStructure`, bare identifier — minimal stamping over its major-0 payload layout gives major 0; the discriminant itself a schema-*minor* vocabulary append), with the matching `OperationKindTag` 28/29 (name-verbatim projection). No existing assignment changed; Section 9.5 restates the per-payload stamping with the pair. Semantics: Operation Catalog Repeat Structures, 0.7.0; re-anchoring row (nearest-surviving-anchor across every anchor site, cascade when none): core Chapter 6 rule table.

Date	Section	Change
October 11, 2026	Operation vocabularies	0.7.0 — Transpose algebra (Push 4a): appends `OperationKind` 30 (`TransposeInterval`: seq⇑(`PitchId`) ‖ two `i32`s) and the matching `OperationKindTag` 30. Every constituent is a major-0 layout, so the payload stamps major 0 and the discriminant is a schema-*minor* vocabulary append — no major 3. Introduces the seq⇑ notation (strictly increasing; a decoder **MUST** reject a duplicate rather than normalize it) and marks 9 (`Transpose`) frozen, its seq↑ being a multiset. Appends two `PreconditionFailureReason` discriminants in the same epoch, 14 (`AcousticRealizationPinned`: a target whose `AcousticRealization::AbsoluteHz` overrides the tuning system, so moving its scale position would move the notehead without moving the sound) and 15 (`TranspositionOutOfRange`: an `alteration` or `octave` driven past its `i8` bound, where the frozen 9 saturates and reports success). No existing assignment changed. Semantics: Operation Catalog TransposeInterval, 0.8.0; algebra: core Chapter 2, Transposition and the Interval Type.
October 11, 2026	Chunk references	0.8.0 — Strict `CompressionAlgorithm` decode (Push 5, P3): `req:binfmt:compression-none-parameter` **MUST** reject a non-zero parameter byte on the `None` discriminant. Ignoring it made the codec non-injective — two byte strings, one value — which every structure embedding a `ChunkRef` inherited; the operation index, having no whole-value re-encode guard, accepted both. Found by the wire-decode fuzzer. Supersedes the core specification's "ignored on read"; no wire layout changed and no conforming writer is affected.
October 11, 2026	Golden anchors	0.9.0 — Ratifies the decode vector corpus (Push 5, P4): `req:binfmt:decode-vectors` and Section 11.1. The cross-implementation decoder test moves out of the About This Companion exclusion list and out of the Golden Anchor Registry's "deferred conformance harness" note: literal byte vectors with accept/reject verdicts now exist (`spec/vectors/decode_vectors.txt`), and a conforming decoder owes them the verdict. Accepting a `reject` vector and normalizing it is *accepting* it. The wire-format fuzzer remains an implementation deliverable. No wire layout changed.

Date	Section	Change
October 11, 2026	Schema evolution / Graph value layouts	0.10.0 — Defines **schema major 3** (Section 9.6, Push 4b tranche 3b-i): `ScoreTuningContext` gains `smufl` (`SmuflVersionRequirement`) and `overrides` (seq(`TuningOverride`)) on the canonical wire, with layouts for the four new leaf types (`SmuflVersion`, `SmuflVersionRequirement`, `TuningScope`, `TuningOverride`) and the total default-filling v2→v3 migration table. `accidental_extensions` is deliberately staged to a later major (no consumer yet) and stays off the wire. Reaches only the acceleration full-`Score` snapshot — no operation payload embeds the tuning context, so the canonical operation layer and its accept-set are untouched (the first data-model major under which a chunk role's max does not move in lockstep with the others); the canonical base and manifest stay major 0, byte-identical. Accept-set: `Snapshot` widens to 3, `OperationEnvelopeBlock` stays at 2. Semantics: core specification Chapter 4, Score Tuning Context / SMuFL Versioning; the staging ruling itself is Push 4b's own (`spec/CONTRACT_PUSH4B_3BI_WIRE.md`).
October 11, 2026	Golden anchors	0.11.0 — Extends the decode vector corpus (Section 11.1) to `epiphany-core`'s score wire, closing the gap §0.9.0 left open and delivering the "future revision" this section's own rationale anticipated: all five representative layouts of Section 5.4 (`RationalTime`, `TimeAnchor`, `Pitch`, `Event`, `Slur`), the schema-major-3 tuning-context leaves and their container, and one whole-`Score` vector per schema major `0`–`3`. Includes the mandatory regression vector for the tranche-3b-i defect (a swapped `smufl`/`overrides` field order, which previously passed round-trip locking and 8/8 conformance). No wire layout changed; no existing vector's bytes moved (`spec/CONTRACT_CORE_DECODE_VECTORS.md`).

Date	Section	Change
October 11, 2026	Operation wire forms	0.12.0 — Genesis tranche G2a (`spec/CONTRACT_GENESIS_G2A_SETTINGS.md`), plus the repair of a G1 omission this document's own contract wrongly declared out of scope. Appends `OperationKind` wire discriminants 31 (`CreateInstrument`, genesis tranche G1 — landed at `3b09595` with no matching entry here), 32 (`SetCanvasLayoutDefaults`), and 33 (`SetSpellingPrecedence`), with their payload layouts, and the matching `OperationKindTag` discriminants 31–33 (name-verbatim projection, as the two most recent prior additions are) — schema-*minor* evolutions under this document's own append-only rules; no existing assignment changed. `CreateInstrument` embeds the full `Instrument` value and is *born at v2* unconditionally (Section 9.5), which the Section 9.4/9.5 "Where the changed fields reach" bullets are corrected to name: neither `Instrument` nor `Canvas.layout_defaults` is snapshot-only any longer, and the Schema Major 2 canonical operation-layer count moves from eight payloads to nine. Both new G2a payloads — `SetCanvasLayoutDefaultsOp` and `SetSpellingPrecedenceOp` — carry only major-0 layouts (neither carried type has ever been versioned as a standalone value), so no accept-set moves: `OperationEnvelopeBlock` stays at 2. The chunk schema *minor* is knowingly still not stamped for any of 24–33 (filed as P13-S14, a retroactive sweep sequenced after this rung). Semantics: Operation Catalog CreateInstrument (retroactive), SetCanvasLayoutDefaults, SetSpellingPrecedence, 0.10.0.

Date	Section	Change
October 11, 2026	Schema Evolution	0.13.0 — G-minor (`spec/CONTRACT_GMINOR_IMPLEMENTATION.md`, `spec/PLAN_GMINOR_SCHEMA_MINOR.md`): implements the minor **MUST** of Section 9.1 for the first time (`binary_format.tex:2330`, unchanged in substance, wired to a real writer-side derivation for the first time). Adds the global additive-epoch derivation (this chapter, What "Additive" Means Here) and repairs the stale 30 next-free-slot narrative this section previously carried, which omitted 30–33 (`TransposeInterval`, `CreateInstrument`, `SetCanvasLayoutDefaults`, `SetSpellingPrecedence`) from both the number and the history; the normative kind and tag tables were already current. No discriminant, wire layout, or accept-set changes: this is a correction to prose and a new writer-side derivation, not a new appended variant.
October 11, 2026	Operation wire forms	0.14.0 — Genesis tranche G2b (`spec/CONTRACT_GENESIS_G2B_TUNING.md`): appends `OperationKind` wire discriminant 34 (`SetTuningContext`) with its payload layout, and the matching `OperationKindTag` discriminant 34 (name-verbatim, following the three most recent prior additions rather than the tag space's older Create→Insert convention) — a schema-*minor* evolution (epoch 10, `spec/PLAN_GMINOR_SCHEMA_MINOR.md` §4) under this document's own append-only rules; no existing assignment changed. The payload carries `epiphany_core::TuningContextSettings`, the **five-field authored subset** of `ScoreTuningContext` — not the full graph type — byte-identical to the existing schema-major-3 wire walk, so that `accidental_extensions` (staged off the wire) cannot diverge between a live-authored session and the same document reloaded from bytes. `SetTuningContext` is the **sole genesis payload born at v3 unconditionally** (Section 9.6), so `OperationEnvelopeBlock`'s accept-set raises 2 → 3 — the first accept-set move since genesis tranche G2a explicitly recorded staying at 2. Semantics: Operation Catalog SetTuningContext, 0.11.0.

Date	Section	Change
October 11, 2026	Operation wire forms	0.15.0 — Genesis tranche G3a (`spec/CONTRACT_GENESIS_G3A_ENTITIES.md`): appends `OperationKind` wire discriminants 35–38 (`CreateStaffGroup`, `CreatePartDefinition`, `CreateAnalysisLayer`, `CreateView`) with their payload layouts, and the matching `OperationKindTag` discriminants 35–38 (name-verbatim, following the two most recent additions rather than the tag space's older Create→Insert convention) — a schema-*minor* evolution (epoch 11, `spec/PLAN_GMINOR_SCHEMA_MINOR.md` §4) under this document's own append-only rules; no existing assignment changed. All four carried types (`StaffGroup`, `PartDefinition`, `AnalysisLayer`, `ViewDefinition`) have never been versioned as standalone values, so no accept-set moves: `OperationEnvelopeBlock` stays at 3 where genesis tranche G2b left it. Semantics: Operation Catalog CreateStaffGroup, CreatePartDefinition, CreateAnalysisLayer, CreateView, 0.12.0.
October 11, 2026	Operation wire forms	0.16.0 — Genesis tranche G3b (`spec/CONTRACT_GENESIS_G3B_MEASURE.md`), the final rung of the genesis operation tranche: appends `OperationKind` wire discriminant 39 (`CreateMeasure`) with its payload layout, and the matching `OperationKindTag` discriminant 39 (name-verbatim, following the four most recent additions rather than the tag space's older Create→Insert convention) — a schema-*minor* evolution (epoch 12, `spec/PLAN_GMINOR_SCHEMA_MINOR.md` §4) under this document's own append-only rules; no existing assignment changed. Also appends three `PreconditionFailureReason` discriminants, 16–18 (`MeasureMeterMismatch`, `MeasureOutOfOrder`, `MeasureOrderUnverifiable`), the same epoch. `Measure` has never been versioned as a standalone value — its only non-scalar field, `TimeAnchor`, has a hand-written codec with no version branching of its own — so `CreateMeasure` is schema major 0 unconditionally and no accept-set moves: `OperationEnvelopeBlock` stays at 3 where genesis tranche G2b left it. This closes the genesis ladder: G1 → G2a → G-minor → G2b → G3a → G3b. Semantics: Operation Catalog CreateMeasure, 0.13.0.

Date	Section	Change
October 11, 2026	The Fixed Header	0.17.0 — The container format major becomes 1, and `format_minor` restarts at 0 (`spec/CONTRACT_FORMAT_EPOCH_MAJOR1.md`). This is the first *container* major boundary, and it is not a change of wire layout: no field moves, no offset changes, and every byte range in the header table is what it was. What changes is what the major *means*. A major-1 container is one whose every base-bearing commit was validated against a supplied reduction authority; major 0 is legacy and **MUST** be decoded deliberately as such rather than rejected. The major is the carrier precisely because the header never changes after creation. A superblock field could not do this job: a legacy container that later commits a validated base would keep carrying whatever its creation stamped, so provenance would be inheritable by a document that never earned it. A minor bump could not do it either — readers ignore minor, and this document's own rules limit a minor change to appending append-safe discriminants, whereas making a previously-valid canonical base newly rejectable is a semantic acceptance change. The full epoch matrix, including the rule that a legacy container may open but **MUST NOT** acquire a canonical base, is normative in the Core Specification, The Container Epoch.
October 11, 2026	Operation wire forms	0.18.0 — X3.1: appends `OperationKind` wire discriminant 40 (`CreateTuplet`) with its payload layout, the length-framed canonical bytes of the carried `Tuplet`, and the matching `OperationKindTag` discriminant 40, name-verbatim — a schema-*minor* evolution (epoch 13) under this document's own append-only rules; no existing assignment changed. `Tuplet` has never been versioned as a standalone value, so `CreateTuplet` is schema major 0 unconditionally and no accept-set moves: `OperationEnvelopeBlock` stays at 3. The value layout is the one the whole-score codec already embeds in the cross-cutting registry, now reachable per value; `TupletRatio` re-validates at decode, so a degenerate ratio does not decode. Semantics: Operation Catalog CreateTuplet, 0.16.0.

Date	Section	Change
October 11, 2026	Operation wire forms	0.19.0 — X3.6: appends `OperationKind` wire discriminants 41 (`SetClef`) and 42 (`SetKeySignature`) with their payload layouts — the staff instance's id, the length-framed canonical bytes of the change's `RationalTime` offset, and an optional length-framed `Clef` or `KeySignature` — and the matching `OperationKindTag` discriminants, name-verbatim: a schema-*minor* evolution (epoch 14) under this document's own append-only rules; no existing assignment changed. Neither value has been versioned standalone, so both kinds are schema major 0 unconditionally and no accept-set moves. The value layouts are those the whole-score codec already embeds in a staff instance's change sequences, now reachable per value; `KeySignature` re-validates its fifths at decode. Semantics: Operation Catalog SetClef and SetKeySignature, 0.17.0.
October 11, 2026	Schema evolution, operation wire forms	0.20.0 — X3c: schema major 4 (Section 9.7). `Tuplet` appends `display`, a `TupletDisplay` of two one-byte discriminants, so `CreateTuplet` (40) is born at v4 unconditionally where it was major 0, and the `OperationEnvelopeBlock` and `Snapshot` accept-sets rise to 4. A `CreateTuplet` written before major 4 is rejected, recognized by its five-field layout and refused by name. The canonical base stays major 0. The decode corpus gains `core.score_v4` and `core.tuplet`, a major-3 score with a tuplet, and a hidden `CreateTuplet` with its pre-major-4 rejection. Semantics: Operation Catalog CreateTuplet, 0.18.0.

Date	Section	Change
October 11, 2026	Schema evolution, operation wire forms	0.21.0 — X4b: schema major 5 (Section 9.8). An event's mark, ornament and grace slots take their types, a marker its kind, a lyric its syllable, text NFC; `LineStyle` and `SpannerKind` append a variant each and `Score` appends `voice_homes`. `CrossCuttingValue` appends 4 Marker and 5 Lyric, `OperationKind` and `OperationKindTag` append 43 `SetVoiceHome`, and `PreconditionFailureReason` appends 19 `SlotOccupied` and 20 `HomeIsOwnStaff`, at epoch 15. Operations whose values hold a new value stamp major 5 by value; the `OperationEnvelopeBlock` and `Snapshot` accept-sets rise to 5. The canonical base stays major 0. The decode corpus gains `core.score_v5`, a refusal of each major-5 value under `core.score_v4`, an event with marks and a grace, an unordered mark set, an ornament set out of order and one repeating a kind, a lyric not in NFC, and a filled `dynamic` slot under both majors. Semantics: Operation Catalog, 0.21.0.